

Machine Learning Workshop | L03

Introduction to Deep Learning

Recap

Supervised Learning

Regression

Linear Regression

Decision Tree

Random Forest

Neural Networks

Classification

Logistic Regression

Naive Bayes

DT

RF

NN

Support Vector Machines

Unsupervised Learning

Clustering

Centroid

Distribution

Density

Hierarchical

Next...

Supervised Learning

Regression

Linear Regression

Decision Tree

Random Forest

Neural Networks

Classification

Logistic Regression

Naive Bayes

DT

RF

NN

Support Vector Machines

Unsupervised Learning

Clustering

Centroid

Distribution

Density

Hierarchical

Deep Learning

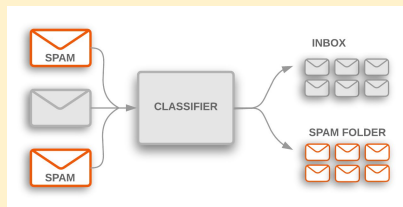
Artificial Intelligence

Any techniques that enables computers to mimic human behaviour



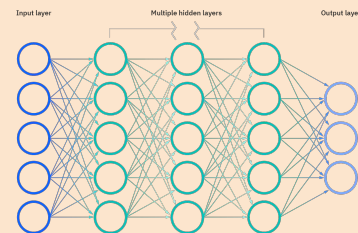
Machine Learning

Ability to learn without explicitly being programmed



Deep Learning

Extract patterns from data using neural networks

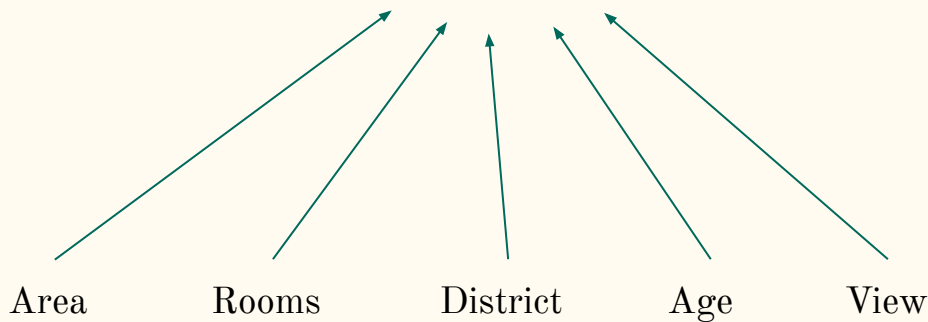


Quick recap : linear and multiple regression

Machine Learning



Assume we want to predict the housing price in HK



label [y]

hidden relationship/
weight [w]

features [X]

$$w_1x_1 + w_2x_2 + \cdots + w_nx_n + b = y$$

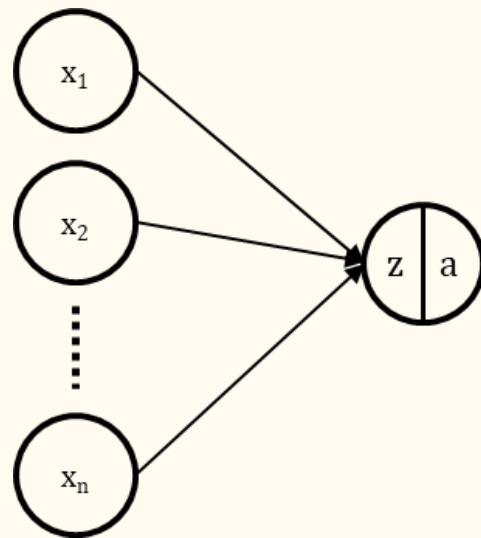
From regression to neural network

$$w_1x_1 + w_2x_2 + \dots + w_nx_n + b = y$$

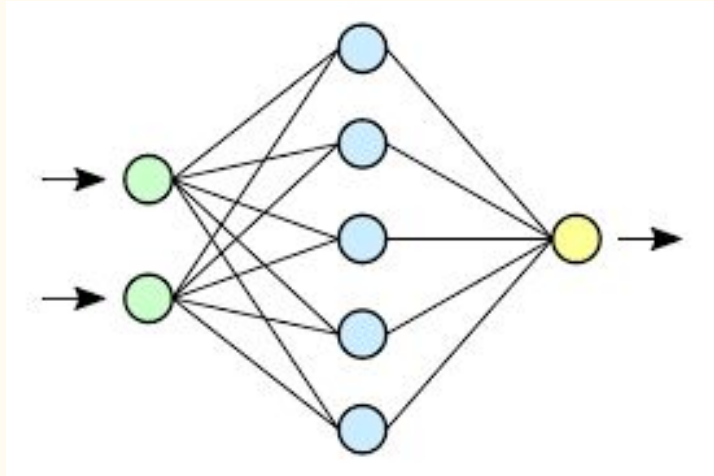
Multiple regression

$$w_1x_1 + w_2x_2 + \dots + w_nx_n + b = z$$
$$g(z) = a$$

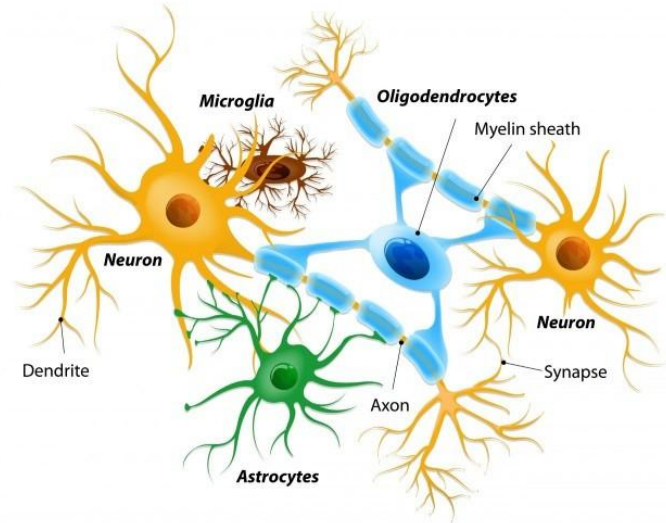
Neural network



Neural network concept



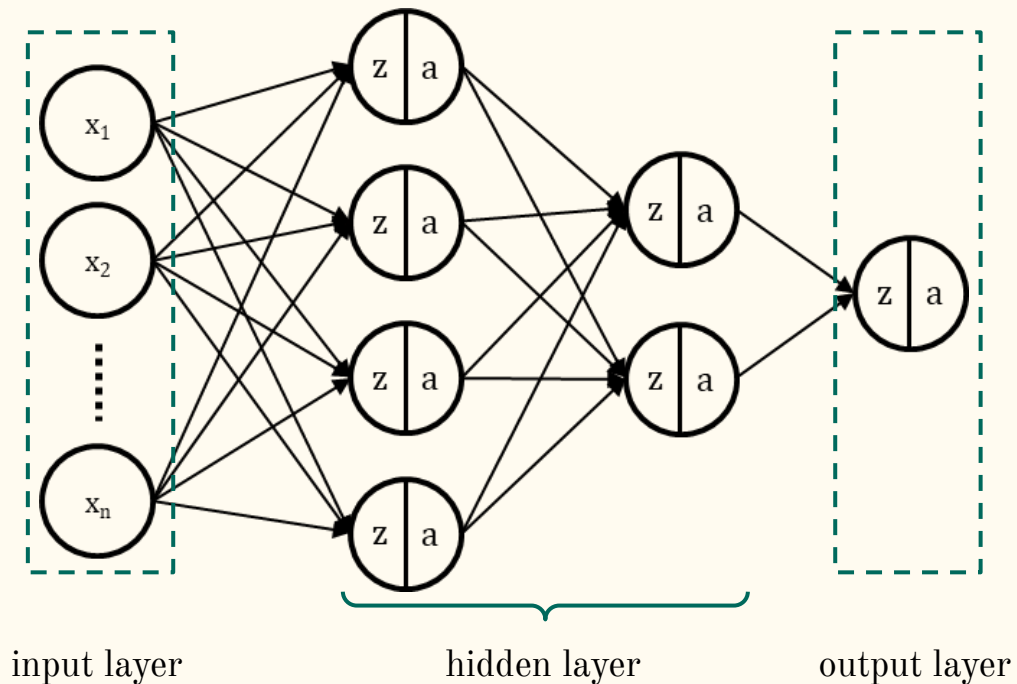
NEURONS AND NEUROGLIAL CELLS



Forward pass

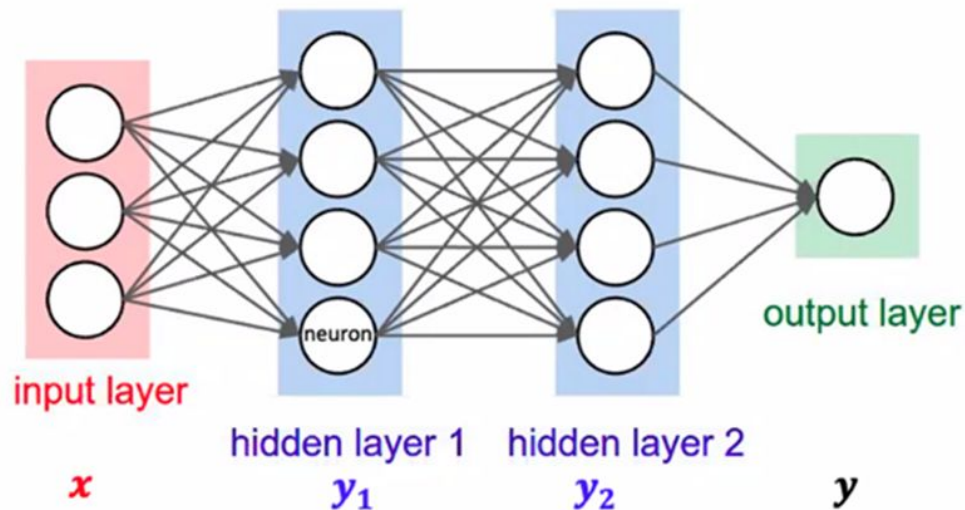
Every node :

- 1) take all the nodes from previous layer as input feature
- 2) multiply(inner product) by weights $\rightarrow z$
- 3) activation function $\rightarrow a$



Artificial Neural Networks: the base of deep learning

Architecture of a 3-layer network



Parameters to solve: weights w and bias b

$4 + 4 + 1 = 9$ neurons (not counting inputs)

$[3 \times 4] + [4 \times 4] + [4 \times 1] = 32$ weights

$4 + 4 = 8$ biases

40 learnable parameters (unknowns)

Weights

$$y_1 = f(w_1 \times x + b_1)$$

Activation function Bias

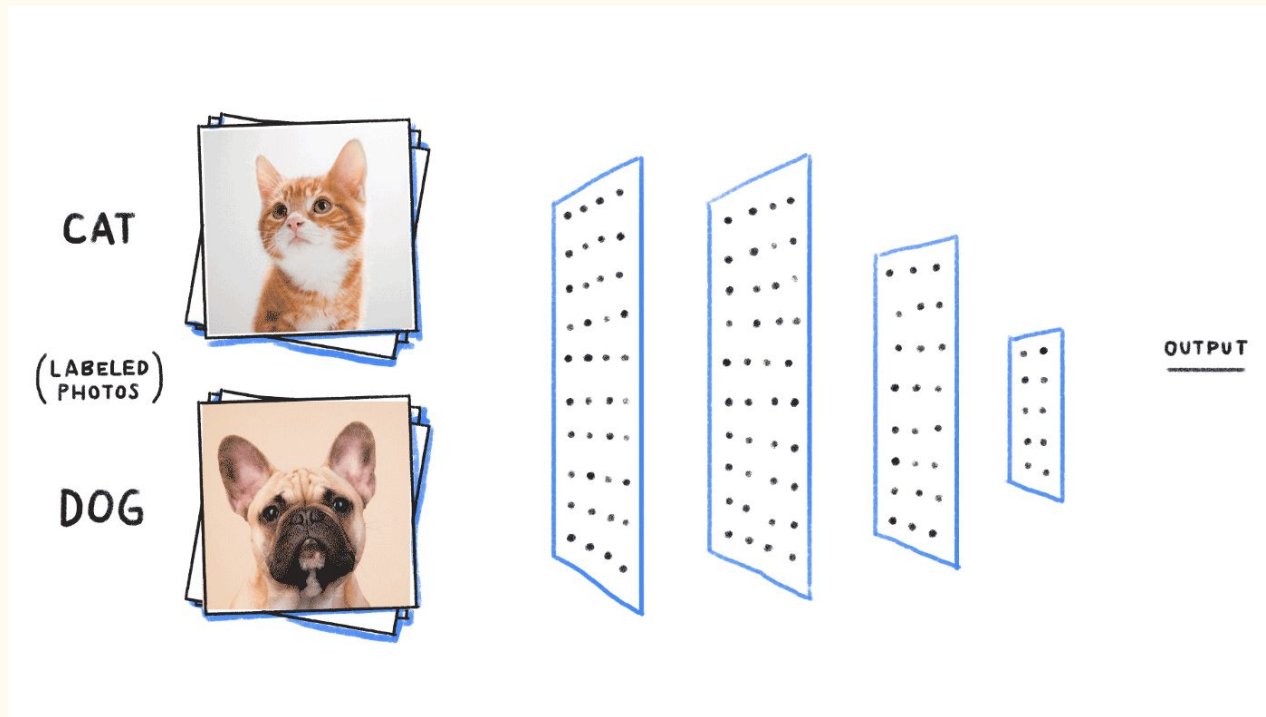
x : 1×3 vector

w_1 : 3×4 matrix

b_1 : 4×1 vector

y_1 : 4×1 vector

Example : image clf



Symbols and notations

$a_1^{[0]}$

upper index : 0th layer (input layer)

lower index : 1st node on 0th layer

$w_{12}^{[1]}$

upper index : proceeding to 1st layer

lower index : 2nd node on 0th layer $\rightarrow$ 1st node on 1st layer

$b_1^{[1]}$

upper index : proceeding to 1st layer

lower index : 1st node on 1st layer

Forward Pass

$$z_1^{[1]} = w_{11}^{[1]} a_1^{[0]} + w_{12}^{[1]} a_2^{[0]} + \dots + w_{1n}^{[1]} a_n^{[0]} + b_1^{[1]}$$

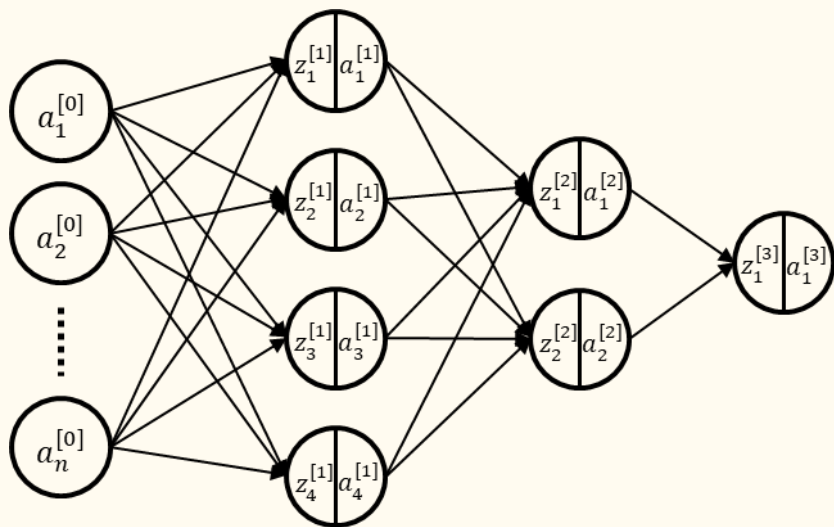
$$a_1^{[1]} = g(z_1^{[1]})$$

$$z_2^{[1]} = w_{21}^{[1]} a_1^{[0]} + w_{22}^{[1]} a_2^{[0]} + \dots + w_{2n}^{[1]} a_n^{[0]} + b_2^{[1]}$$

$$a_1^{[2]} = w_{11}^{[2]} a_1^{[1]} + w_{12}^{[2]} a_2^{[1]} + w_{13}^{[2]} a_3^{[1]} + w_{14}^{[2]} a_4^{[1]} + b_1^{[2]}$$

$$z_{11}^{[3]} = w_{11}^{[3]} a_1^{[2]} + w_{21}^{[3]} a_2^{[2]} + b_1^{[3]}$$

$$a_1^{[3]} = g(z_{11}^{[3]}) = \hat{y}$$



matrix representations of Forward Pass

$$\mathbf{z}^{[1]} = \mathbf{w}^{[1]} \mathbf{a}^{[0]} + \mathbf{b}^{[1]}$$

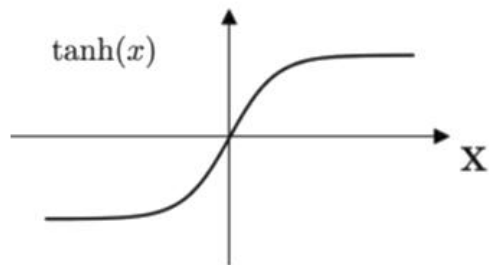
Aim : find the relationships between nodes (w and b)

$$\begin{bmatrix} z_1^{[1]} \\ z_2^{[1]} \\ z_3^{[1]} \\ z_4^{[1]} \end{bmatrix} = \begin{bmatrix} w_{11}^{[1]} & w_{12}^{[1]} & \dots & w_{1n}^{[1]} \\ w_{21}^{[1]} & w_{22}^{[1]} & \dots & w_{2n}^{[1]} \\ \vdots & \vdots & \dots & \vdots \\ w_{41}^{[1]} & w_{42}^{[1]} & \dots & w_{4n}^{[1]} \end{bmatrix} \times \begin{bmatrix} a_1^{[0]} \\ a_2^{[0]} \\ \vdots \\ a_n^{[0]} \end{bmatrix} + \begin{bmatrix} b_1^{[1]} \\ b_2^{[1]} \\ b_3^{[1]} \\ b_4^{[1]} \end{bmatrix}$$

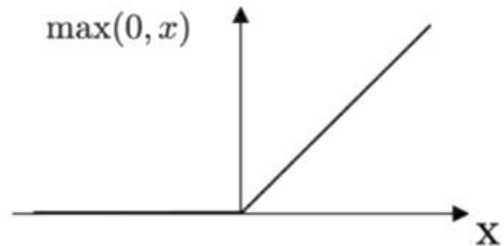
$4 \times 1 \qquad \qquad 4 \times n \qquad \qquad n \times 1 \qquad 4 \times 1$

Activation functions

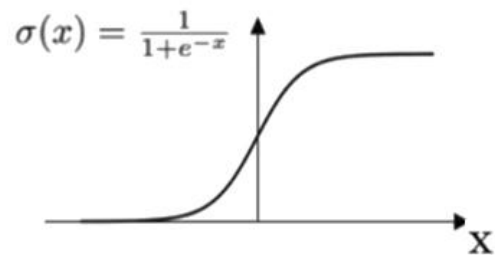
Tanh



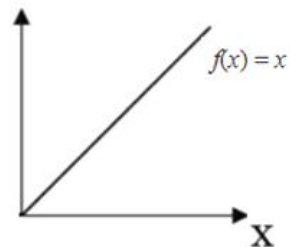
ReLU



Sigmoid



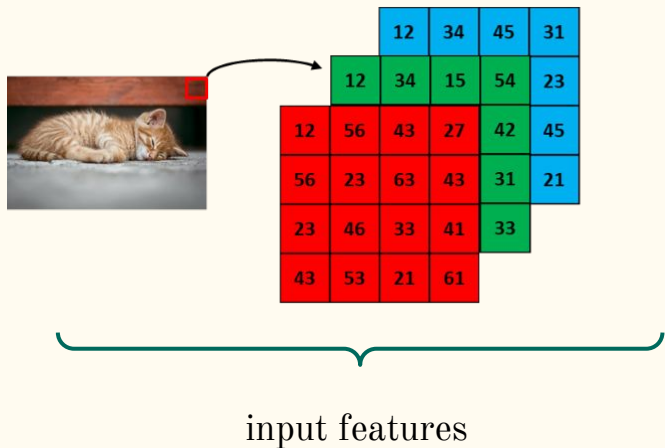
Linear



Example : [Image clf] computer vision

32 x 32 pixel image $\rightarrow$ 32 x 32 x 3 (RGB layer)

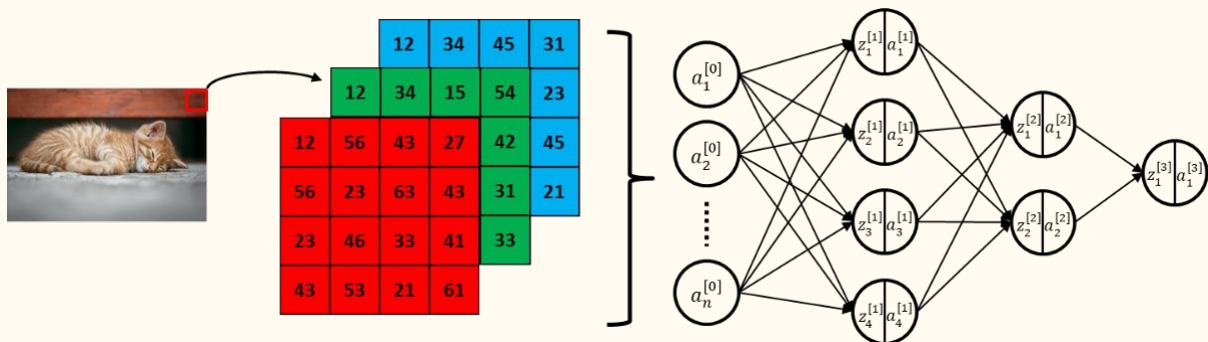
color depth : 6 bit $\rightarrow$ input feature range : 0 - 63 (64 colors)



Example : [Image clf] forward pass

Neural Network : input layer x $1 \rightarrow$ hidden layer $x2 \rightarrow$ output layer $x1$

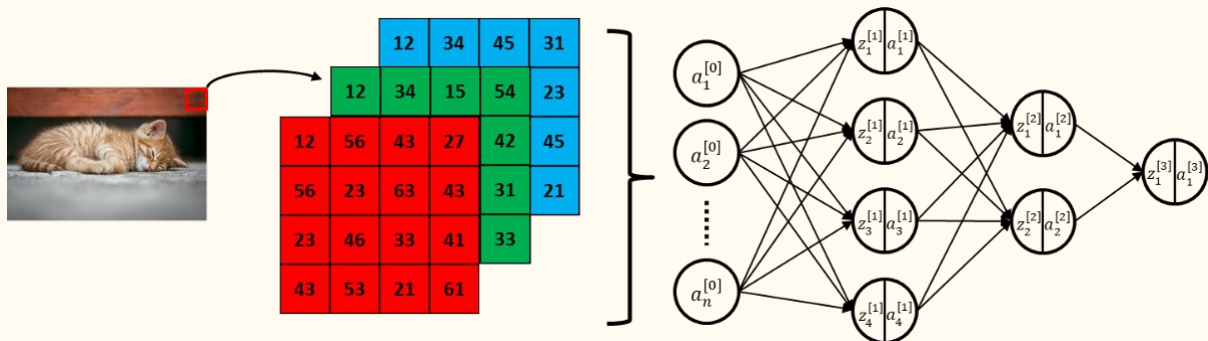
last hidden layer (2 neurons | cat OR non-cat) $\rightarrow$ output layer (1 neuron)



Example : [Image clf] forward pass

Activation function : ReLU (hidden layer) + Sigmoid (output layer)

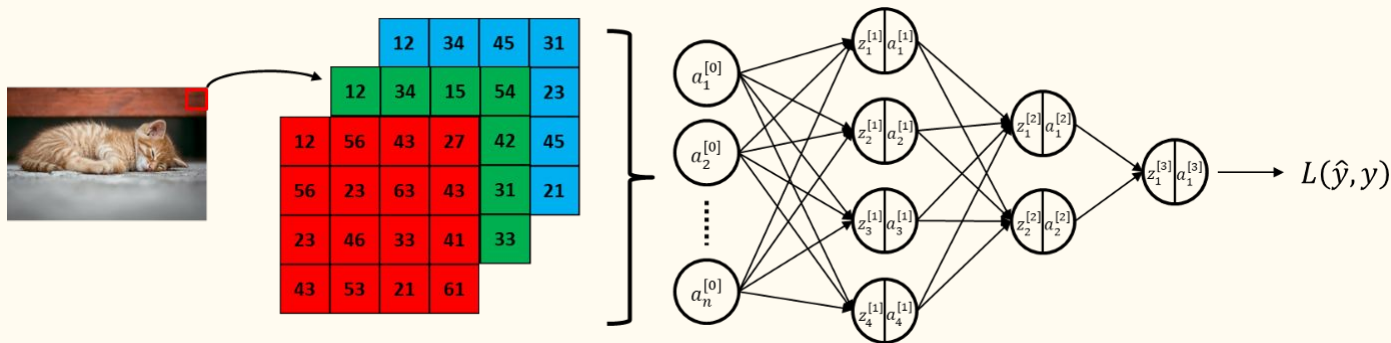
Sigmoid : Binary clf output layer (0 - 1)



Example : [Image clf] Loss function

Loss function L : evaluate the accuracy of our prediction

L : compare label and prediction $\rightarrow$ try different w and $b \rightarrow$ minimize L



Loss function

Loss function : difference between prediction and ground truth

We use different loss function for different scenario

Classification : cross-entropy

$$L(\hat{y}, y) = -y \log \hat{y} - (1 - y) \log(1 - \hat{y})$$

(We will understand more next class, but we take it for granted for now.)

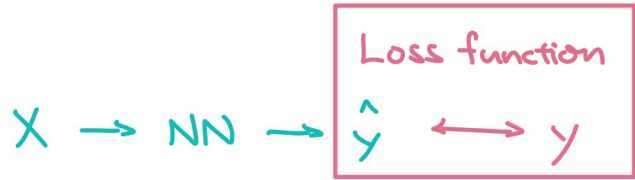
Next class : Shannon Entropy $\rightarrow$ Cross Entropy $\rightarrow$ KL divergence)

Back Propagation

Loss Function $\rightarrow$ try different w / b $\rightarrow$ minimize the loss function

How do we update the parameters from results : BP

Forward Pass $\rightarrow$



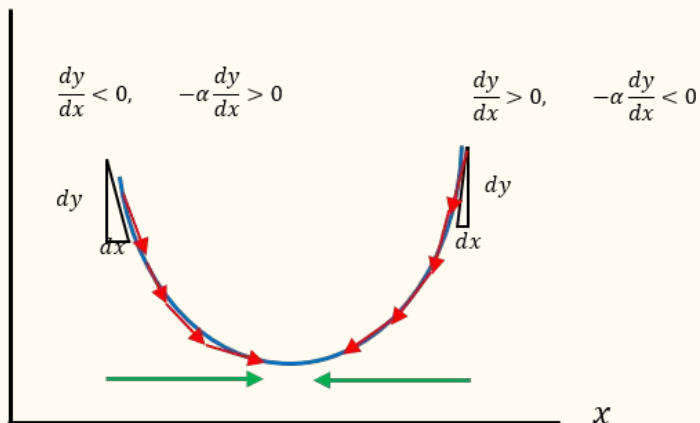
Back propagation

- Gradient Descent
- Update parameters

Gradient descent

Loss Function $\rightarrow$ try different w / b $\rightarrow$ minimize the loss function

$$y = f(x)$$



1. Take Gradient of the Loss Function
2. Pick random values for the parameters
3. Plug the parameters value into the gradient
4. Step sizes : step size = slope x learning rate
5. New parameters = Old parameter - step size

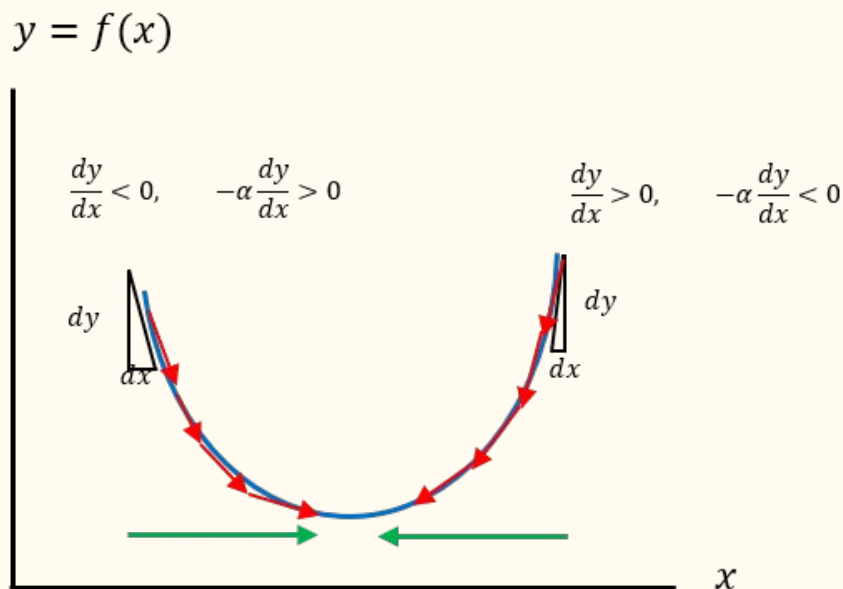
Gradient descent

$$L(\hat{y}, y) = L(x, w^{[1]}, b^{[1]}, w^{[2]}, b^{[2]}, w^{[3]}, b^{[3]}, y)$$

$$w^{[l]} := w^{[l]} - \alpha \frac{\partial L}{\partial w^{[l]}}$$

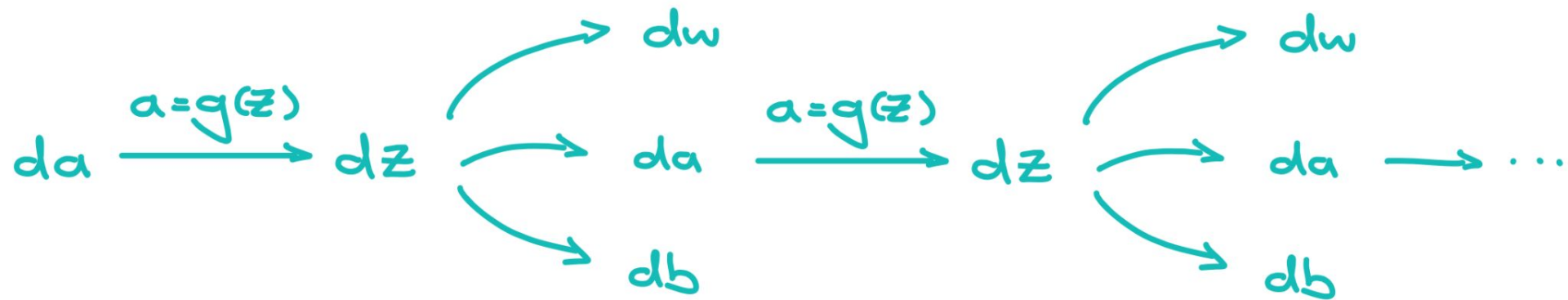
$$b^{[l]} := b^{[l]} - \alpha \frac{\partial L}{\partial b^{[l]}}$$

alpha : learning rate



Back propagation big picture

We will neglect the numerator of partial derivatives for simplicity



Back propagation big picture

We will neglect the numerator of partial derivatives for simplicity



BP chain rule

$$\frac{\partial L}{\partial \hat{y}} = d\hat{y} = da^{[3]} = -\frac{y}{\hat{y}} + \frac{1-y}{1-\hat{y}} = -\frac{y}{a^{[3]}} + \frac{1-y}{1-a^{[3]}} = -\frac{y}{g(z^{[3]})} + \frac{1-y}{1-g(z^{[3]})}$$

$$g(z) = \text{sigmoid}(z) = \frac{1}{1+e^{-z}}$$

$$g'(z) = \frac{-1}{(1+e^{-z})^2} \times (-e^{-z}) = \frac{1+e^{-z}-1}{(1+e^{-z})^2} = \frac{1}{1+e^{-z}} - \frac{1^2}{(1+e^{-z})^2} = g(z) - g(z)^2 = g(z)(1-g(z))$$

$$\frac{\partial L}{\partial z^{[3]}} = dz^{[3]} = da^{[3]} \frac{da^{[3]}}{dz^{[3]}} (\text{chain rule}) = da^{[3]} g'(z^{[3]})$$

$$= -y(1-g(z^{[3]})) + (1-y)g(z^{[3]}) = -y + yg(z^{[3]}) + g(z^{[3]}) - yg(z^{[3]}) = g(z^{[3]}) - y$$

We are trying to calculate the partial derivative of a and z for the last layer

BP chain rule

$$\frac{\partial L}{\partial w^{[3]}} = dw^{[3]} = \frac{\partial L}{\partial z^{[3]}} \frac{dz^{[3]}}{dw^{[3]}} = dz^{[3]} \times a^{[2]} (\text{chain rule} + \text{partial derivative})$$

$$\frac{\partial L}{\partial b^{[3]}} = db^{[3]} \frac{\partial L}{\partial z^{[3]}} \frac{dz^{[3]}}{db^{[3]}} = dz^{[3]}$$

$$\frac{\partial L}{\partial a^{[2]}} = da^{[2]} \frac{\partial L}{\partial z^{[3]}} \frac{dz^{[3]}}{da^{[3]}} = dz^{[3]} \times w^{[3]}$$

z is a function of w, a, b

so we take partial derivative respectively

Iterations

One complete iteration :

Input $\rightarrow$ (Forward Pass) $\rightarrow$ Output $\rightarrow$ Loss Function $\rightarrow$ (BP | GD) $\rightarrow$ update b, w

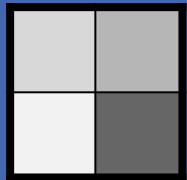
Use the new b, w to proceed the next iteration:

Input $\rightarrow$ (Forward Pass) $\rightarrow$ Output $\rightarrow$ Loss Function $\rightarrow$ (BP | GD) $\rightarrow$ update b, w

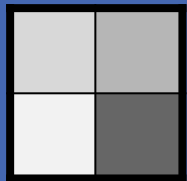
[The loss function should be smaller than the previous iteration]

Repeat until the loss function is minimized (the step in GD becomes extremely small)

A four pixel camera



Categorize images



solid



vertical



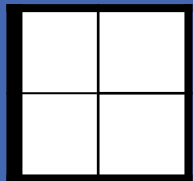
diagonal



horizontal



Categorize images



solid



vertical



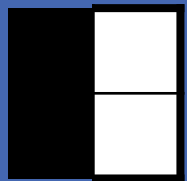
diagonal



horizontal



Categorize images



solid



vertical



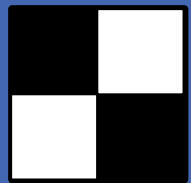
diagonal



horizontal



Categorize images



solid



vertical



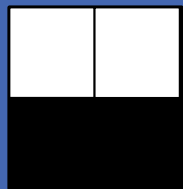
diagonal



horizontal



Simple rules can't do it



solid



vertical



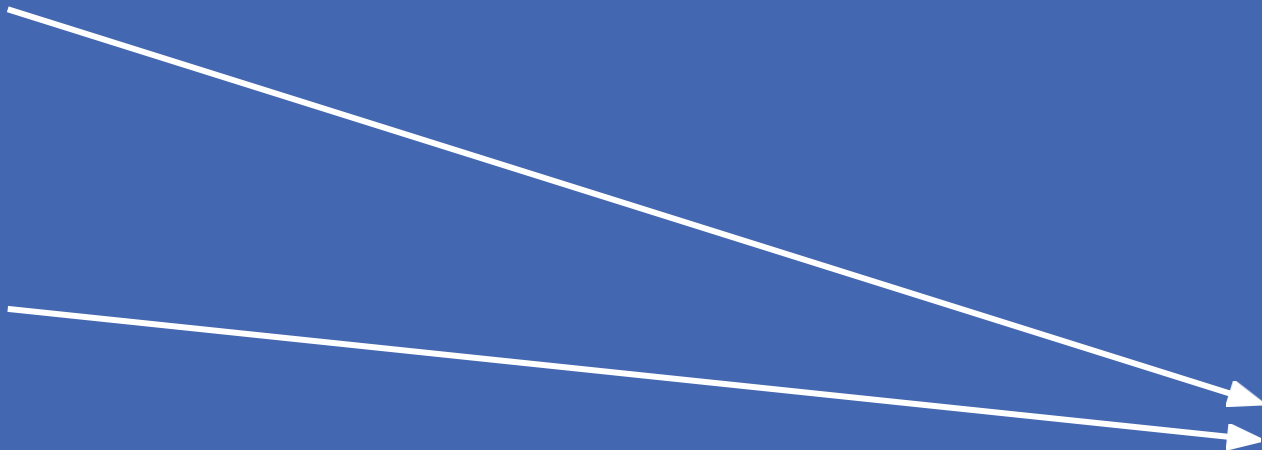
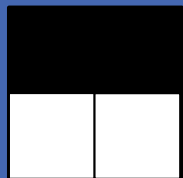
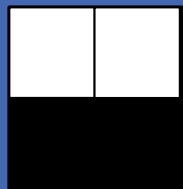
diagonal



horizontal



Simple rules can't do it



solid



vertical



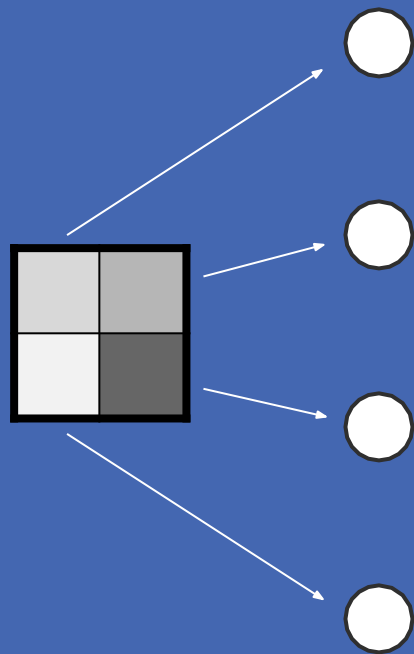
diagonal



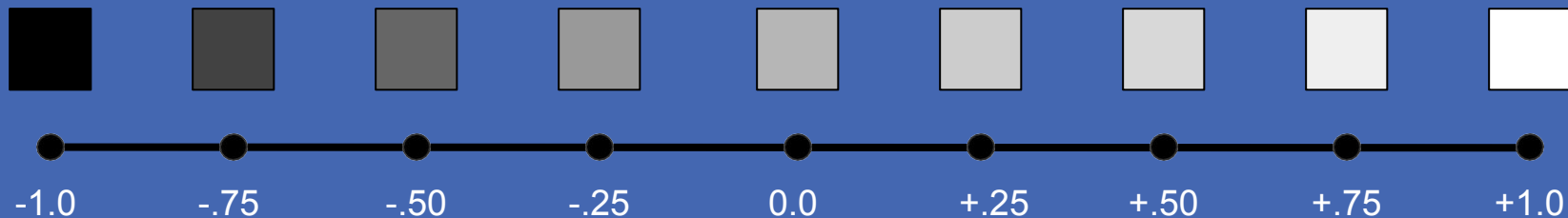
horizontal



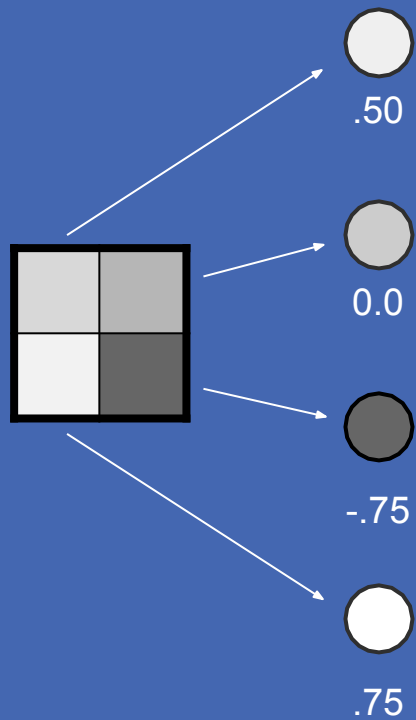
Input neurons



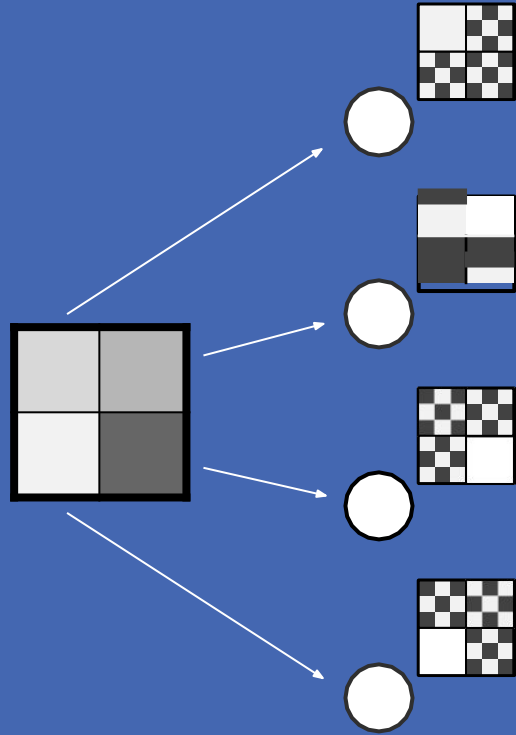
Pixel brightness



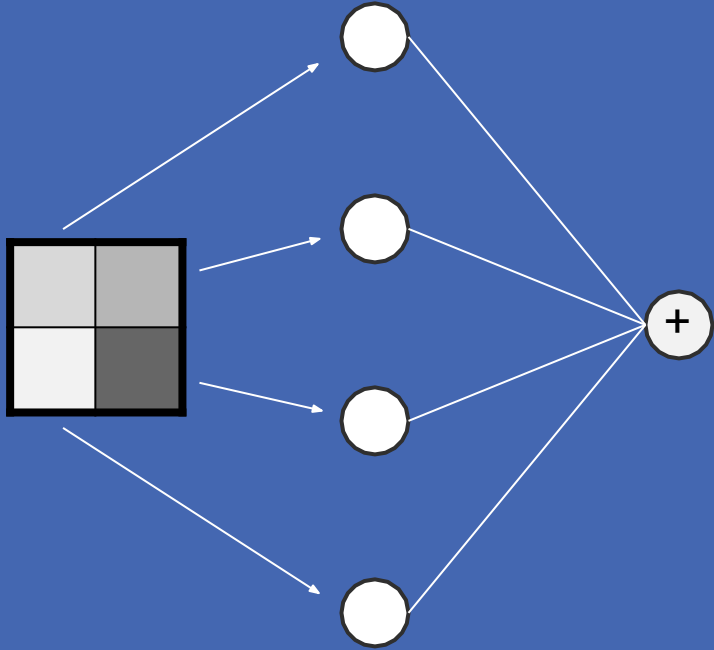
Input vector



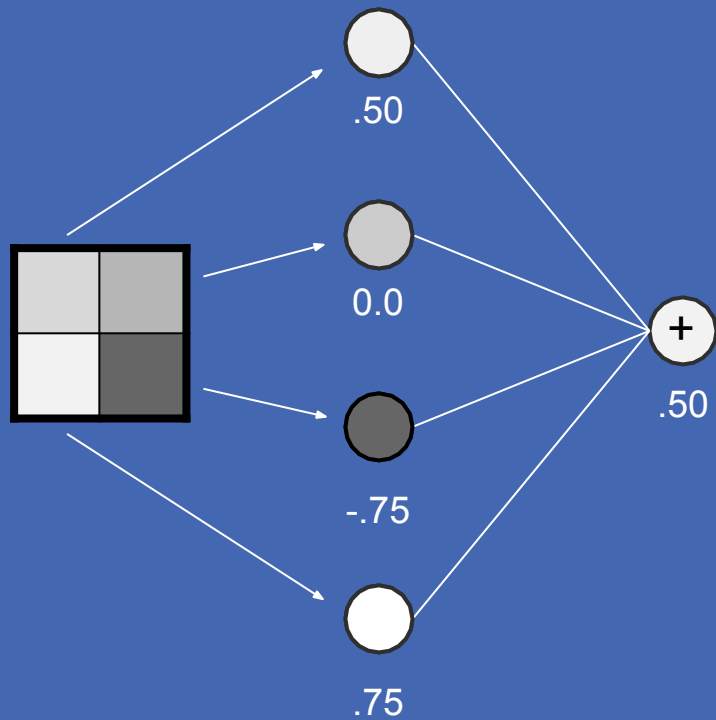
Receptive fields



A neuron

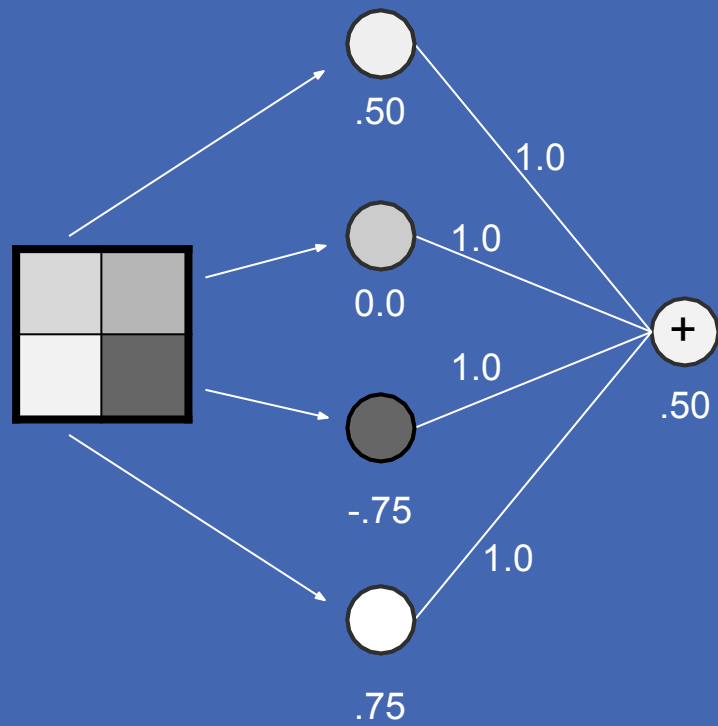


Sum all the inputs



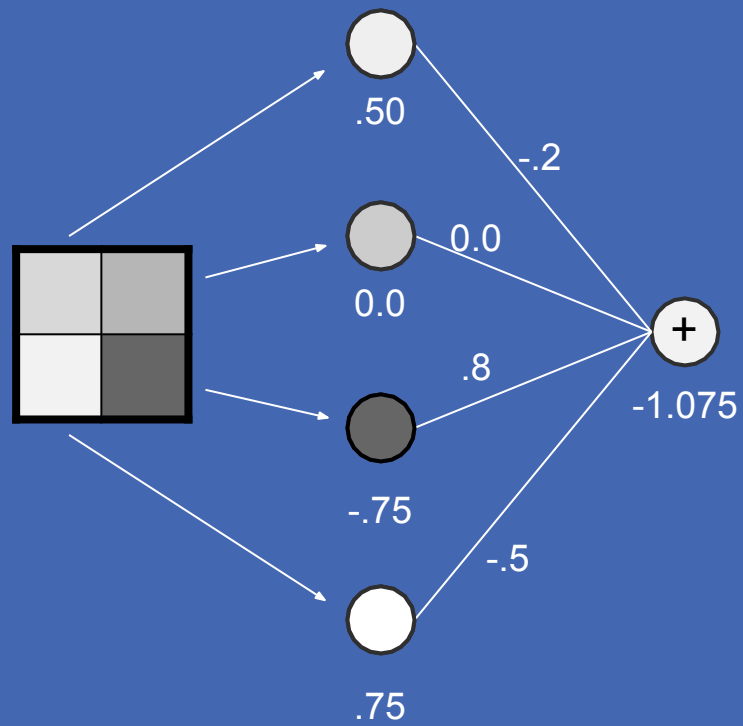
$$\begin{array}{r} .50 \\ 0.00 \\ -.75 \\ + \quad .75 \\ \hline .50 \end{array}$$

Weights



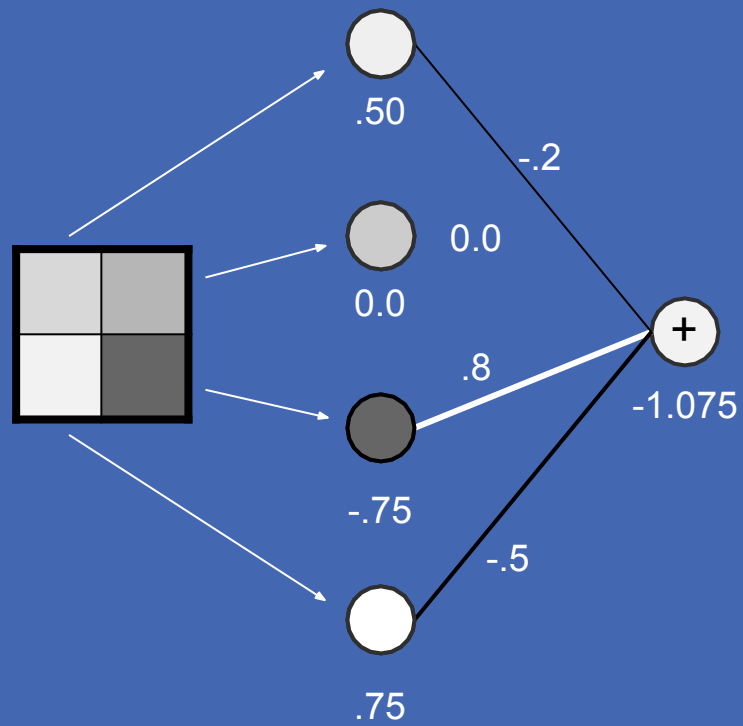
$$\begin{array}{r} .50 \times 1.0 \\ 0.00 \times 1.0 \\ -.75 \times 1.0 \\ + \quad .75 \times 1.0 \\ \hline .50 \end{array}$$

Weights



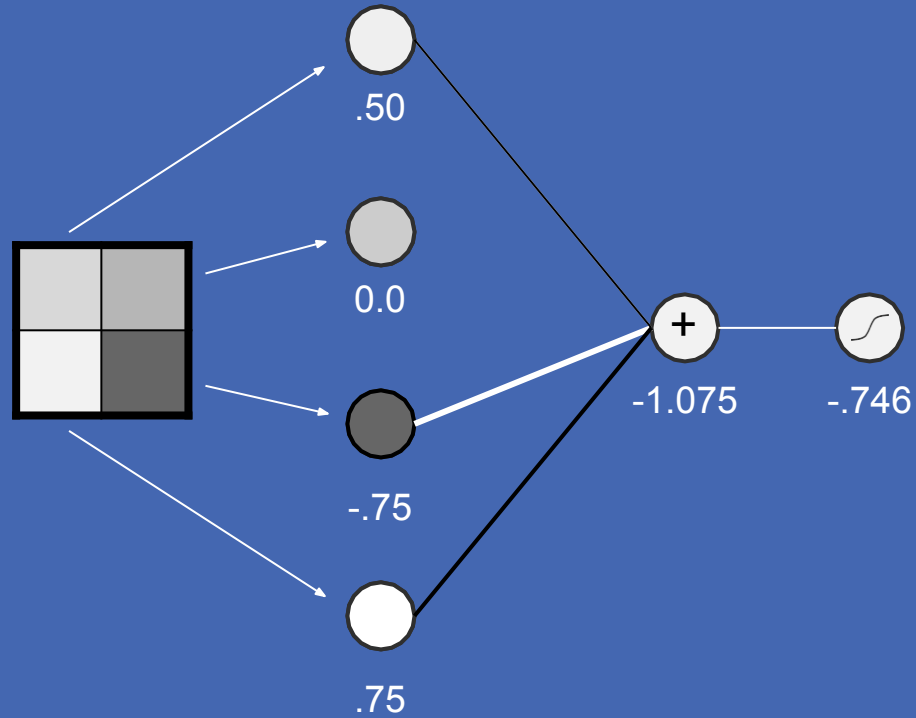
$$\begin{array}{r} .50 \times -.2 \\ 0.00 \times 0.0 \\ -.75 \times .8 \\ + .75 \times -.5 \\ \hline -1.075 \end{array}$$

Weights

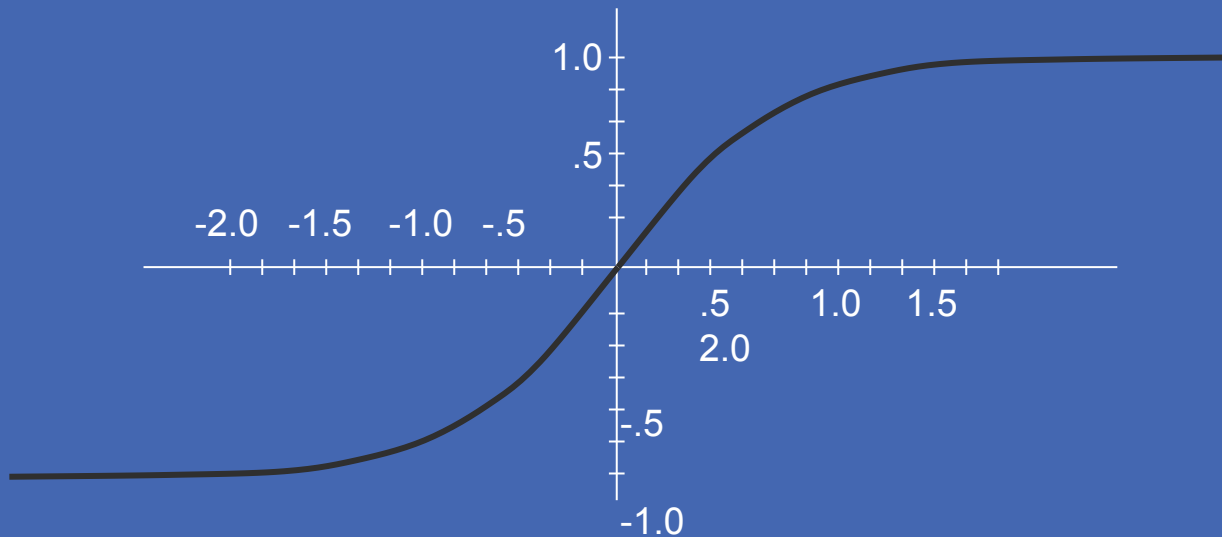


$$\begin{array}{r} .50 \times -.2 \\ 0.00 \times 0.0 \\ -.75 \times .8 \\ + .75 \times -.5 \\ \hline -1.075 \end{array}$$

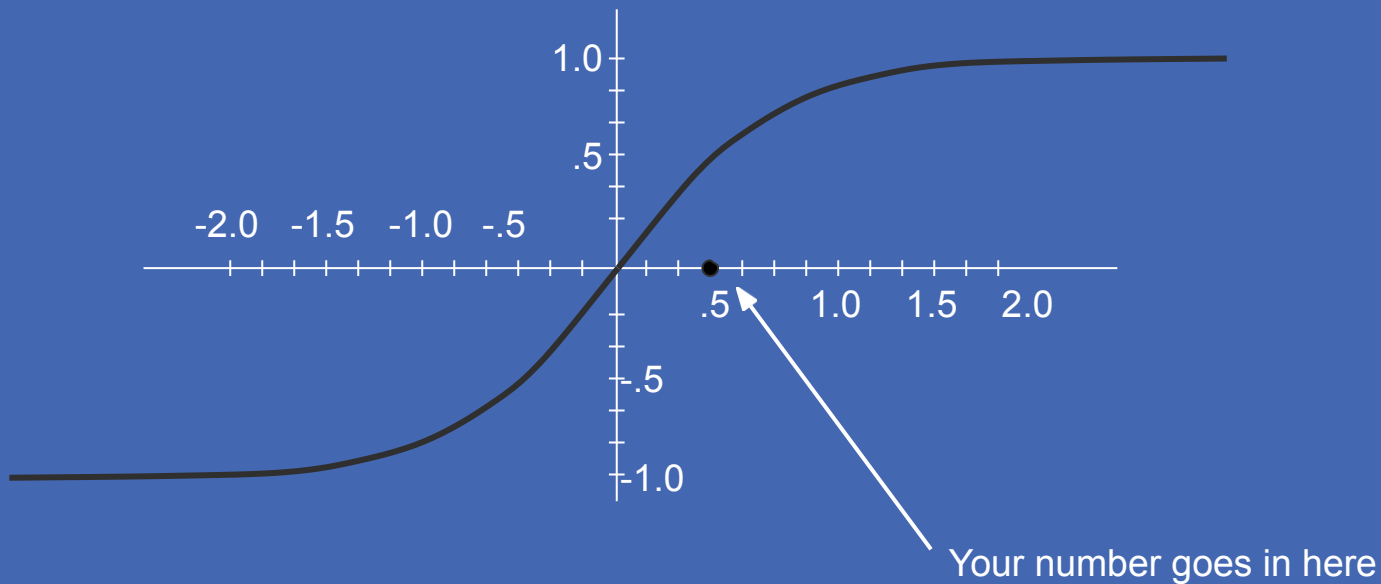
Squash the result



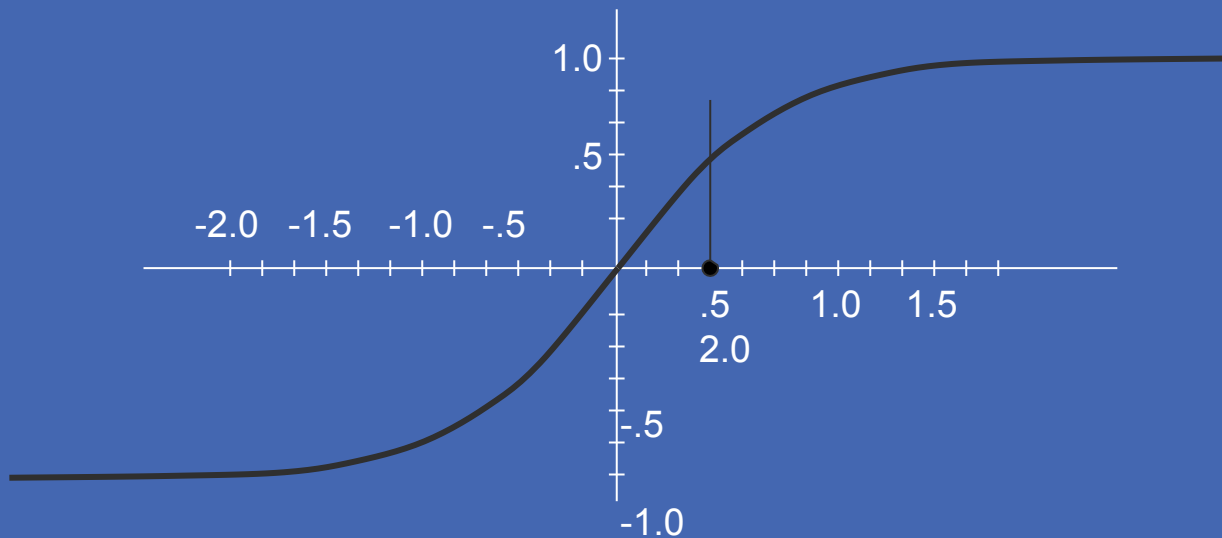
Hyperbolic tangent (tanh) squashing function



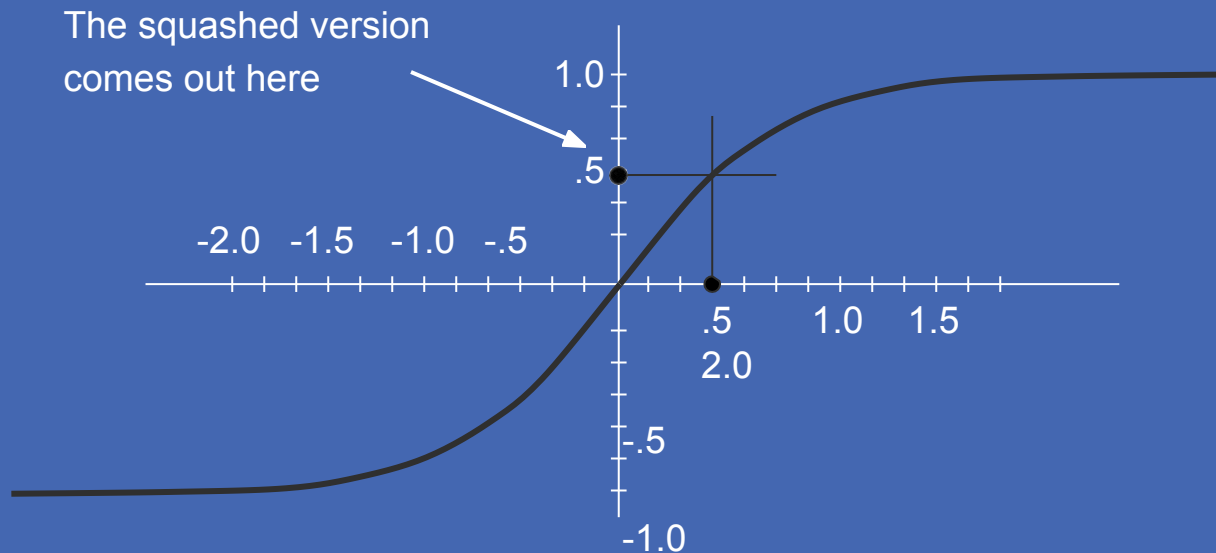
tanh squashing function



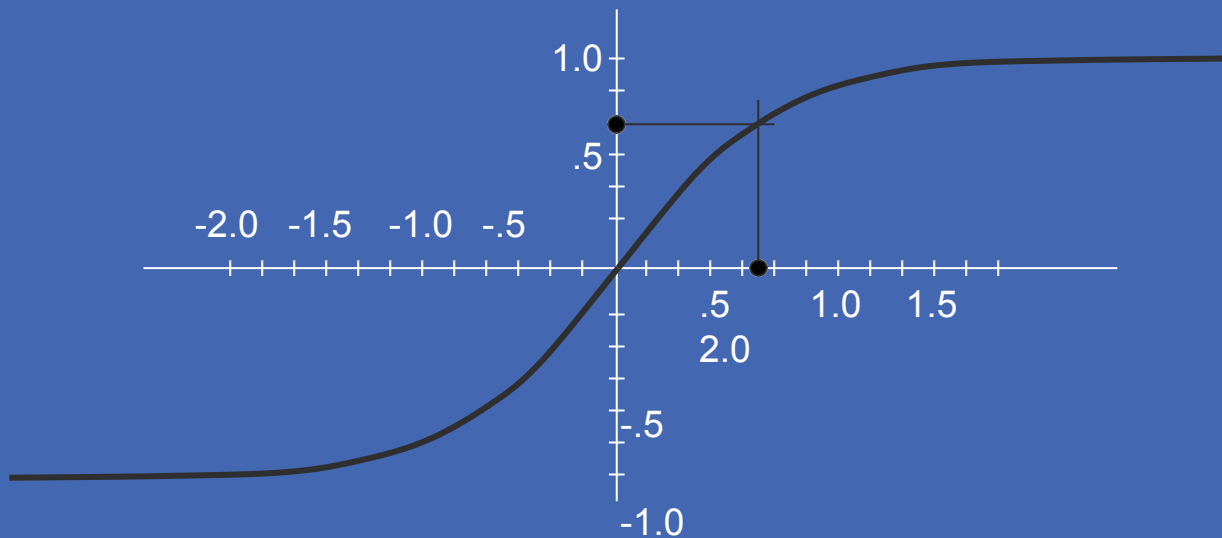
tanh squashing function



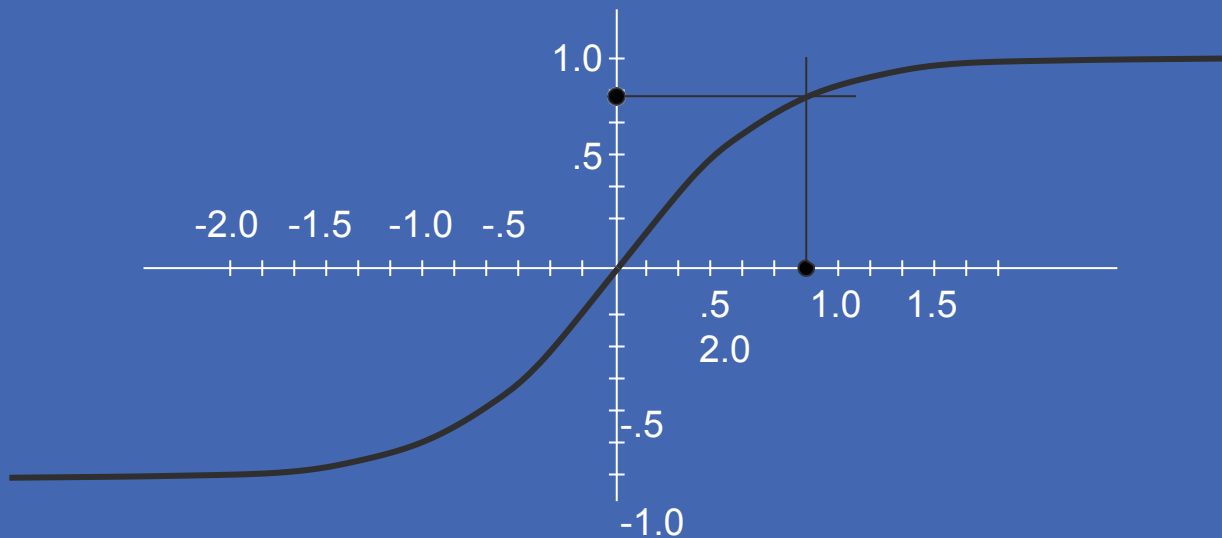
tanh squashing function



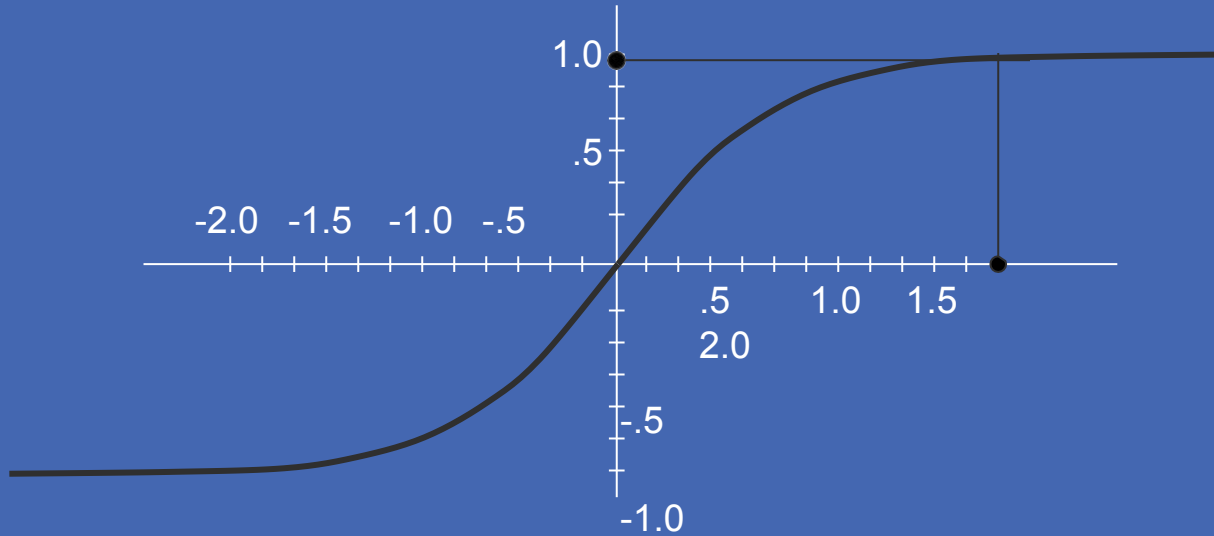
tanh squashing function



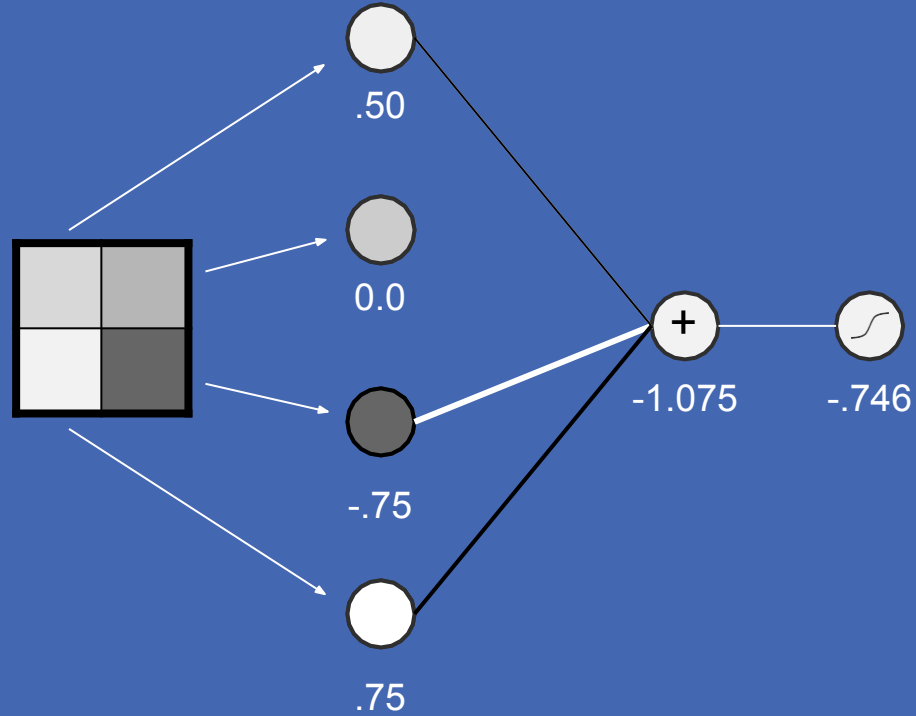
tanh squashing function



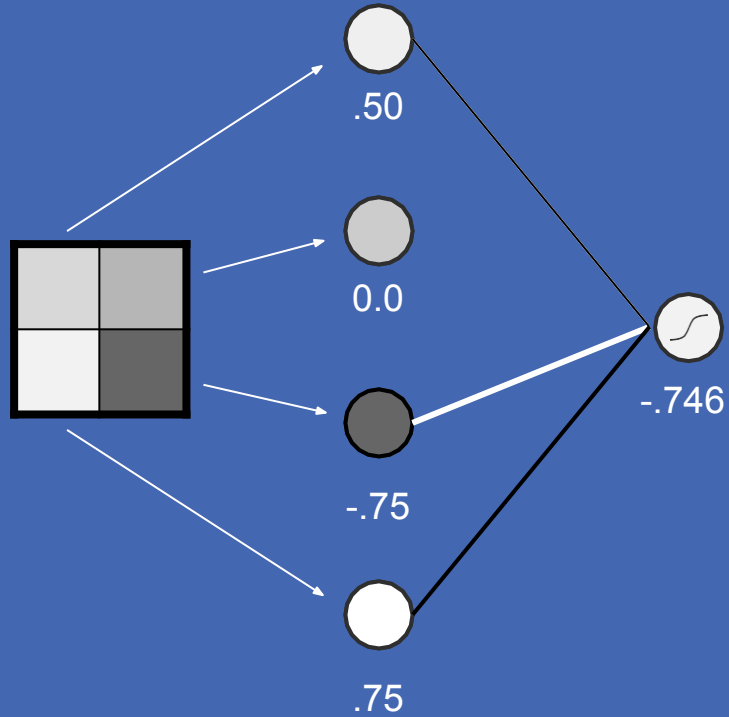
No matter what you start with, the answer stays between -1 and 1.



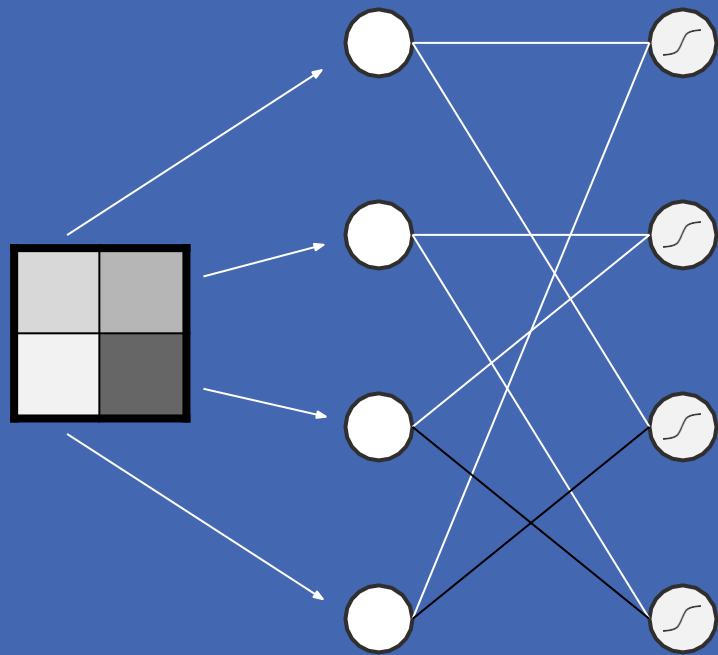
Squash the result



Weighted sum-and-squash neuron

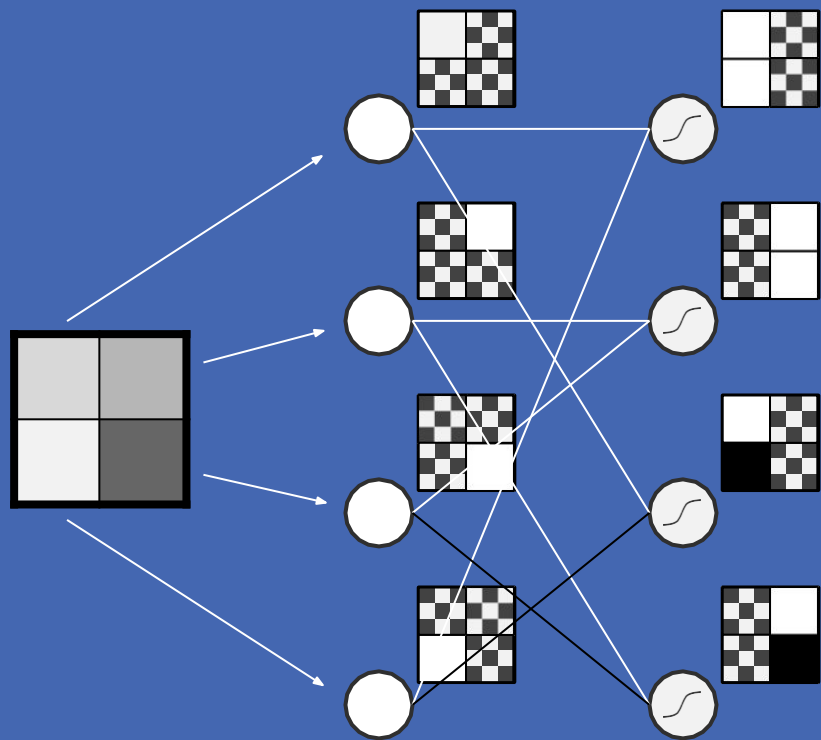


Make lots of neurons, identical except for weights

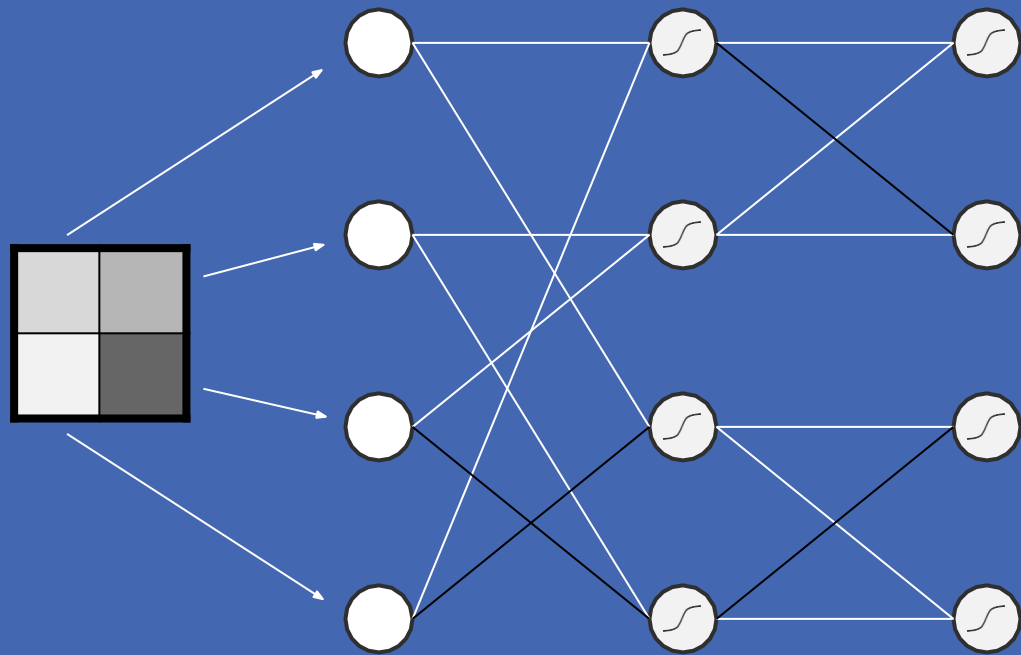


To keep our picture
clear, weights will either
be
1.0 (white)
-1.0 (black) or
0.0 (missing)

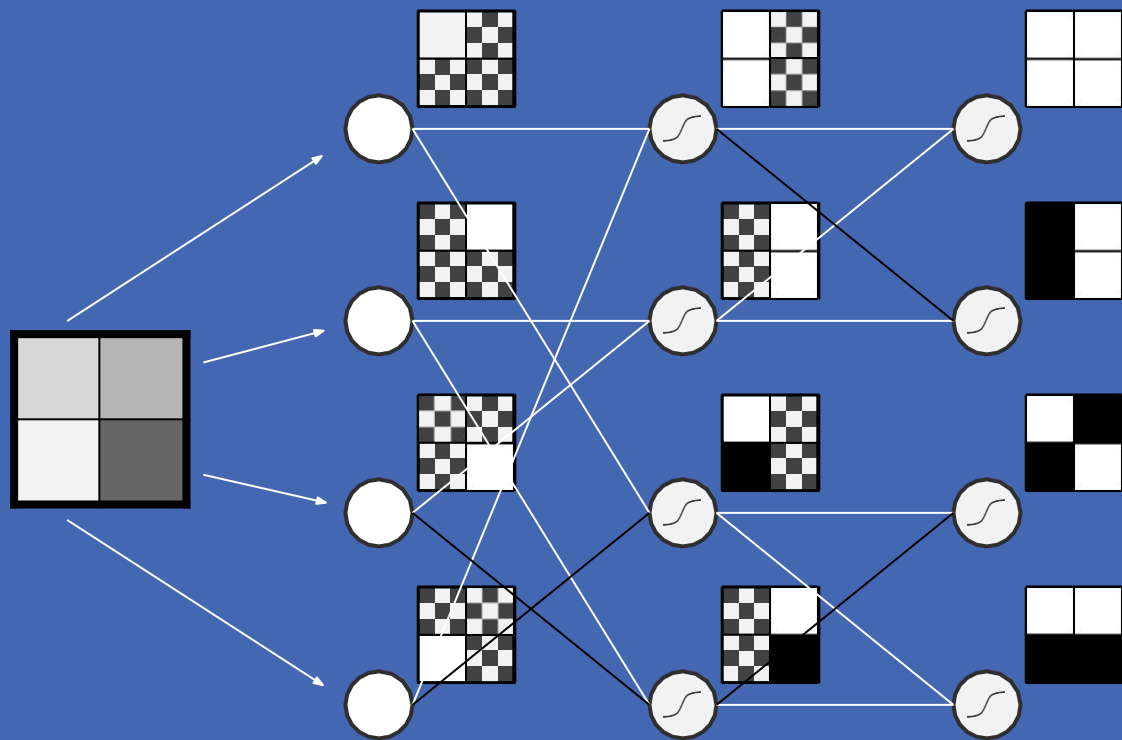
Receptive fields get more complex



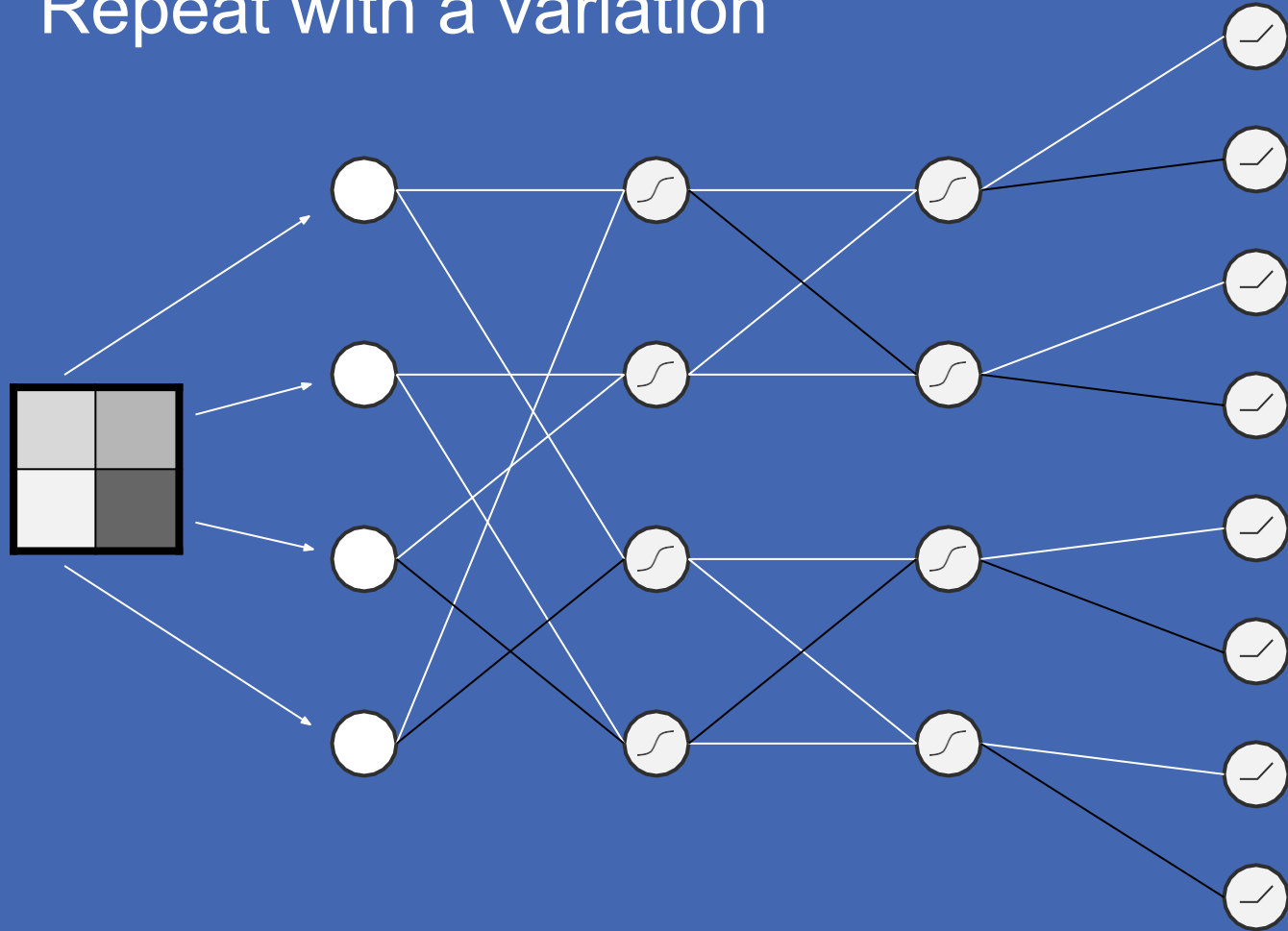
Repeat for additional layers



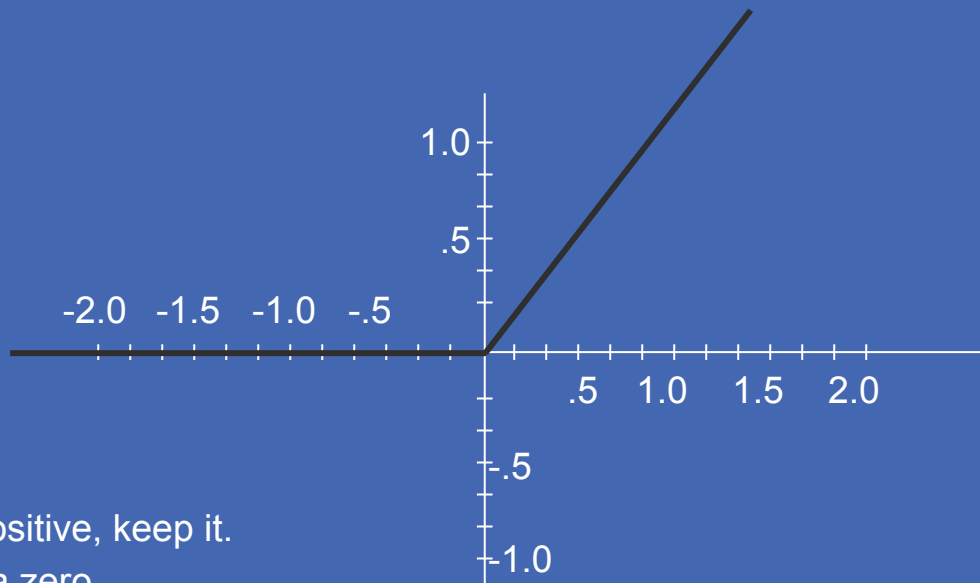
Receptive fields get still more complex



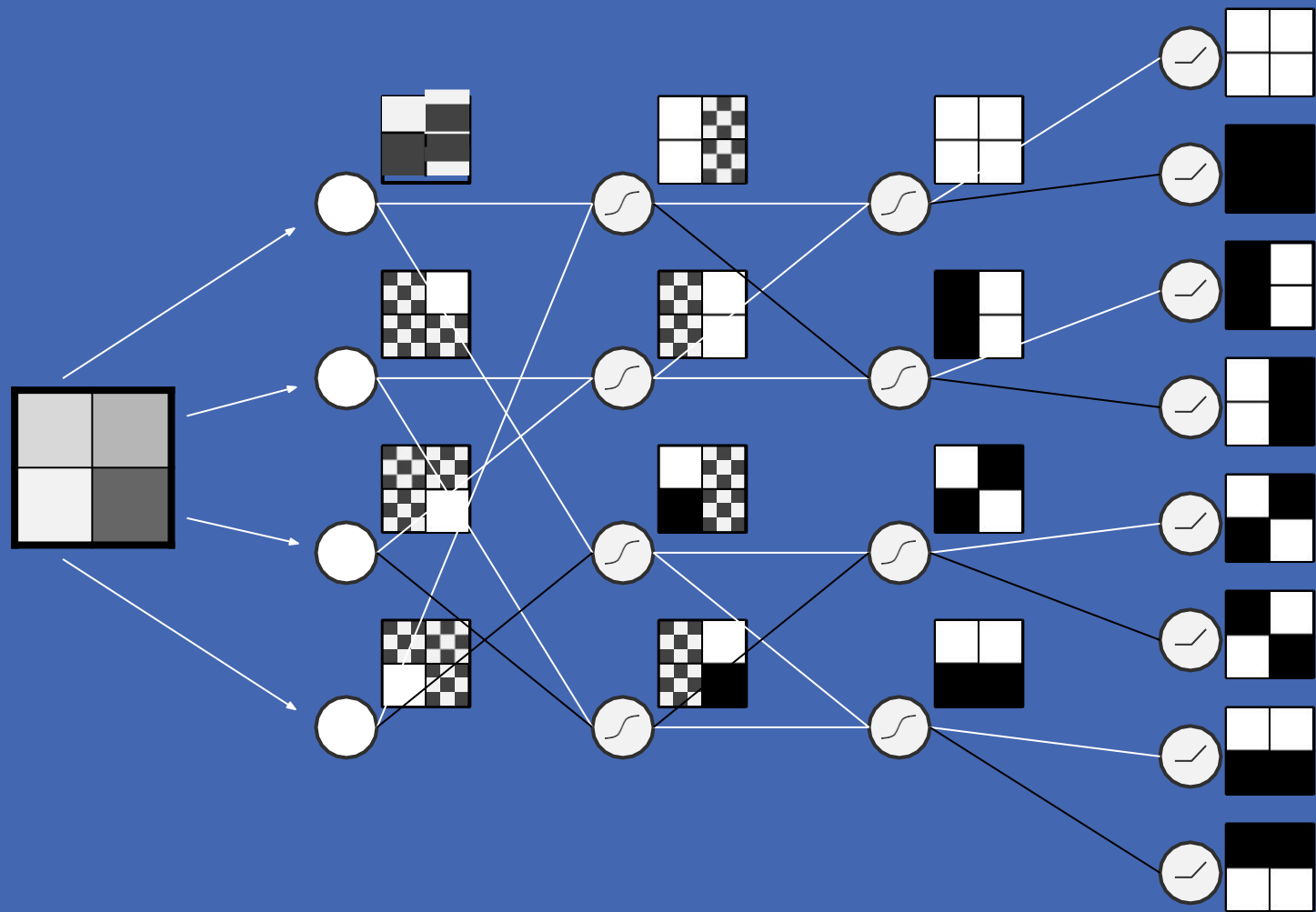
Repeat with a variation



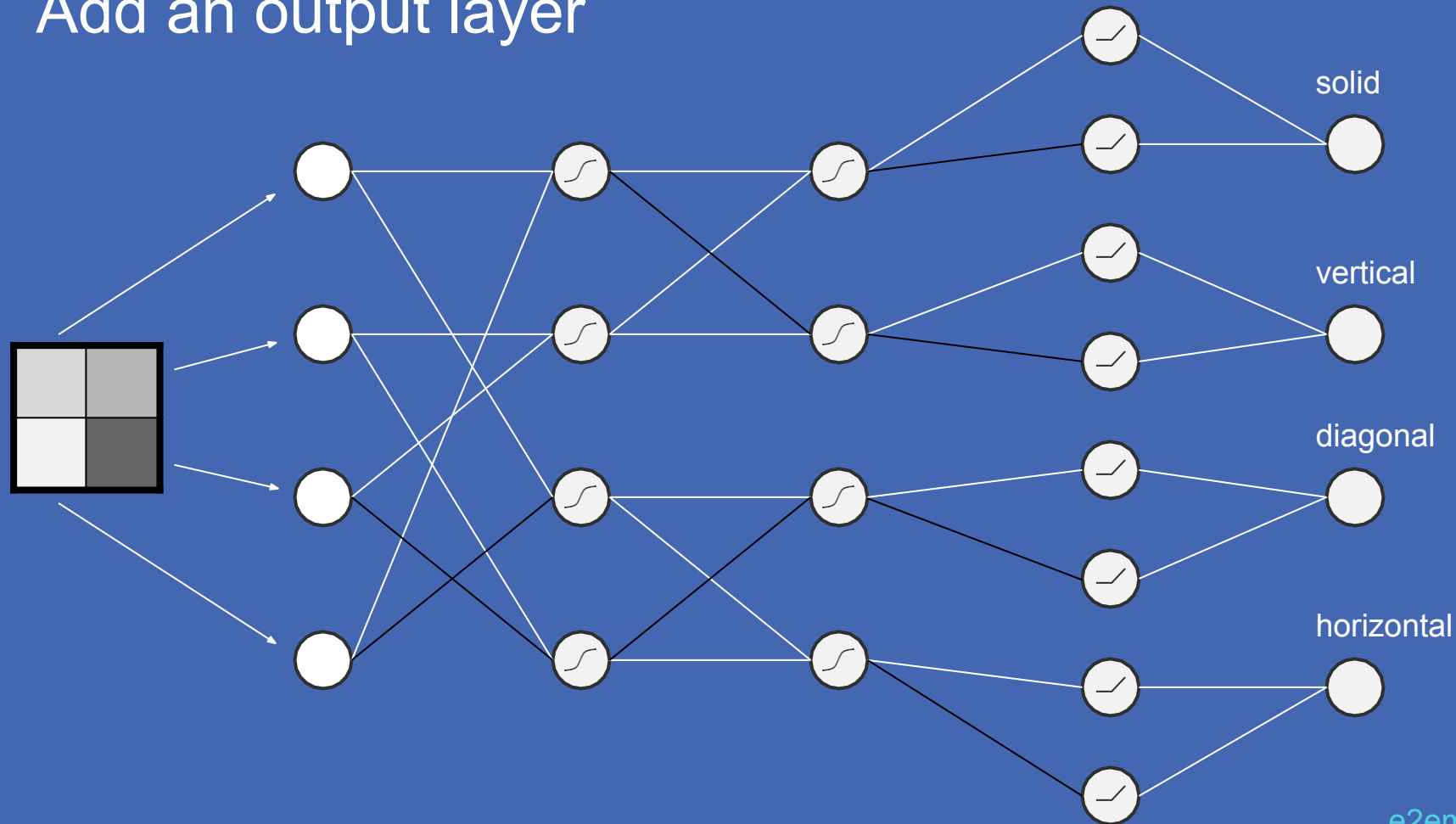
Rectified linear units (ReLUs)

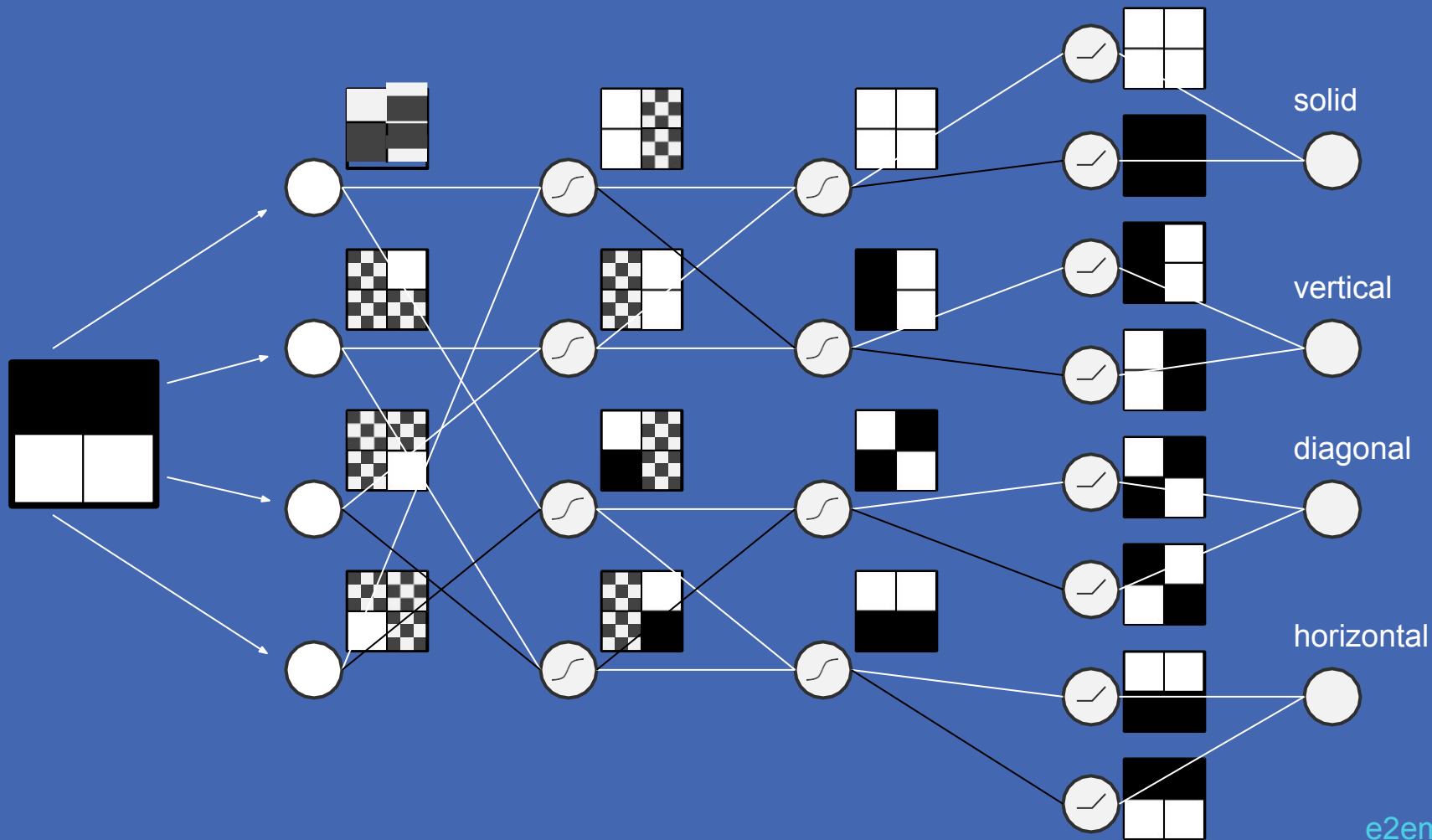


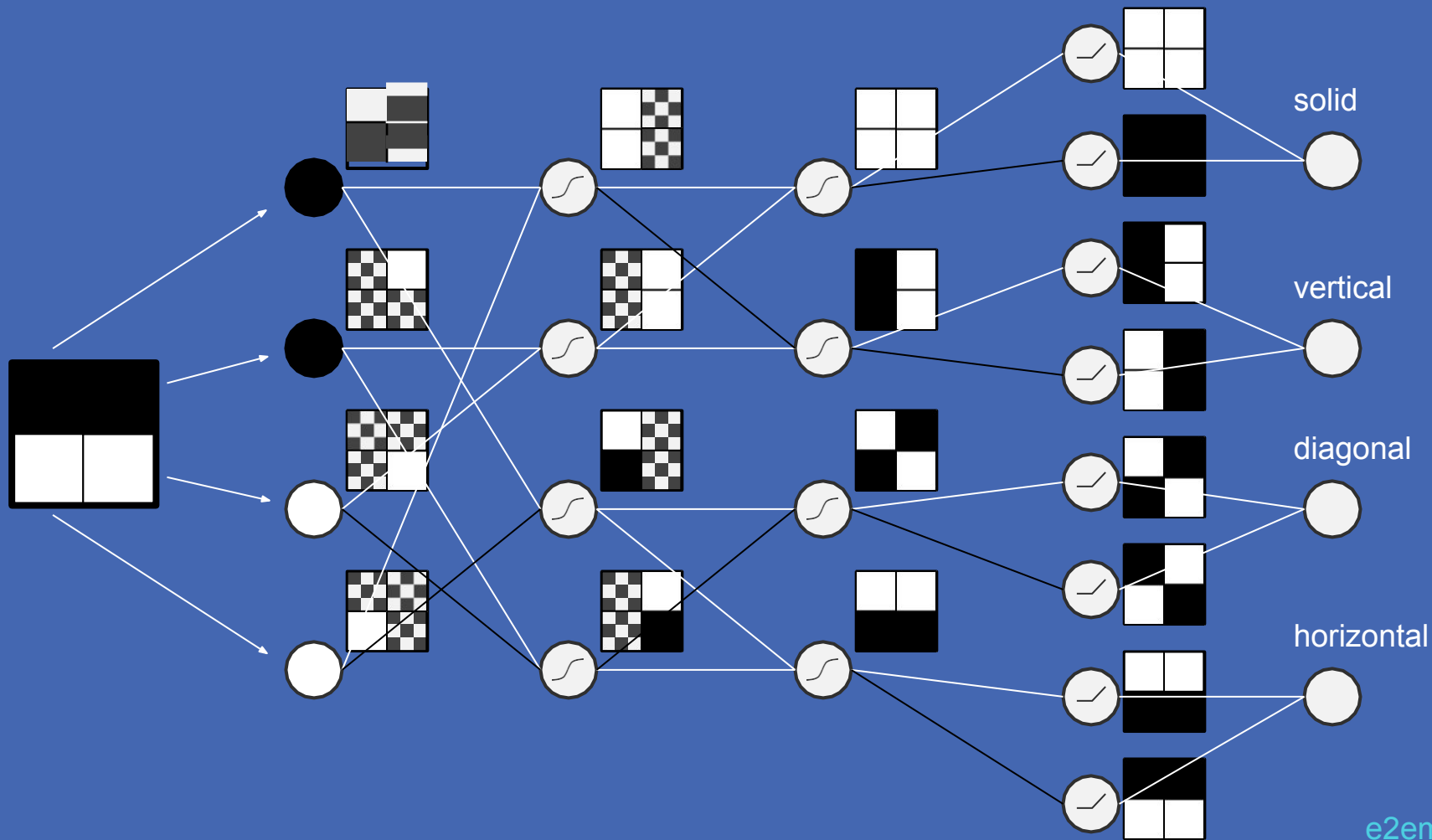
If your number is positive, keep it.
Otherwise you get a zero.

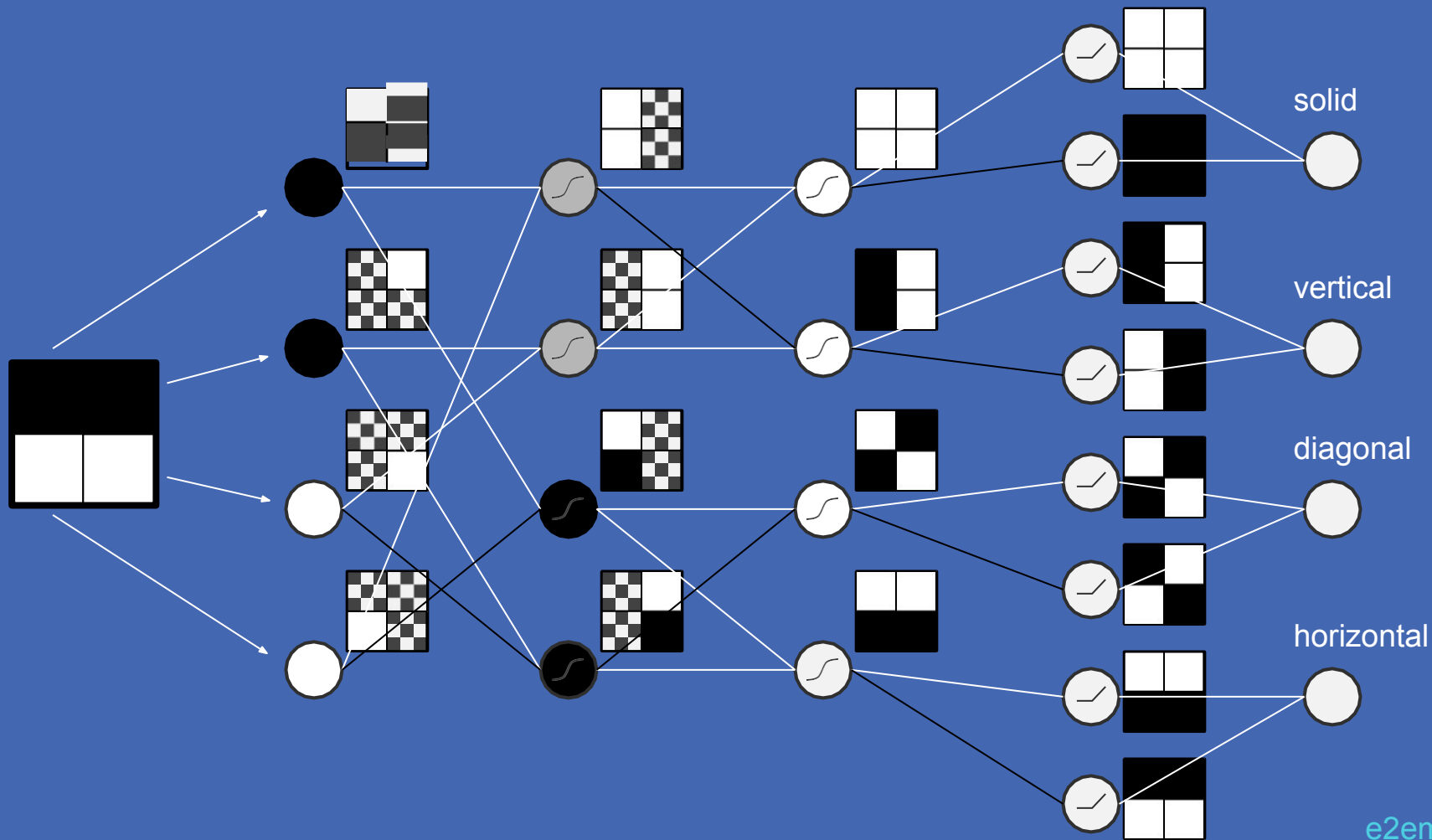


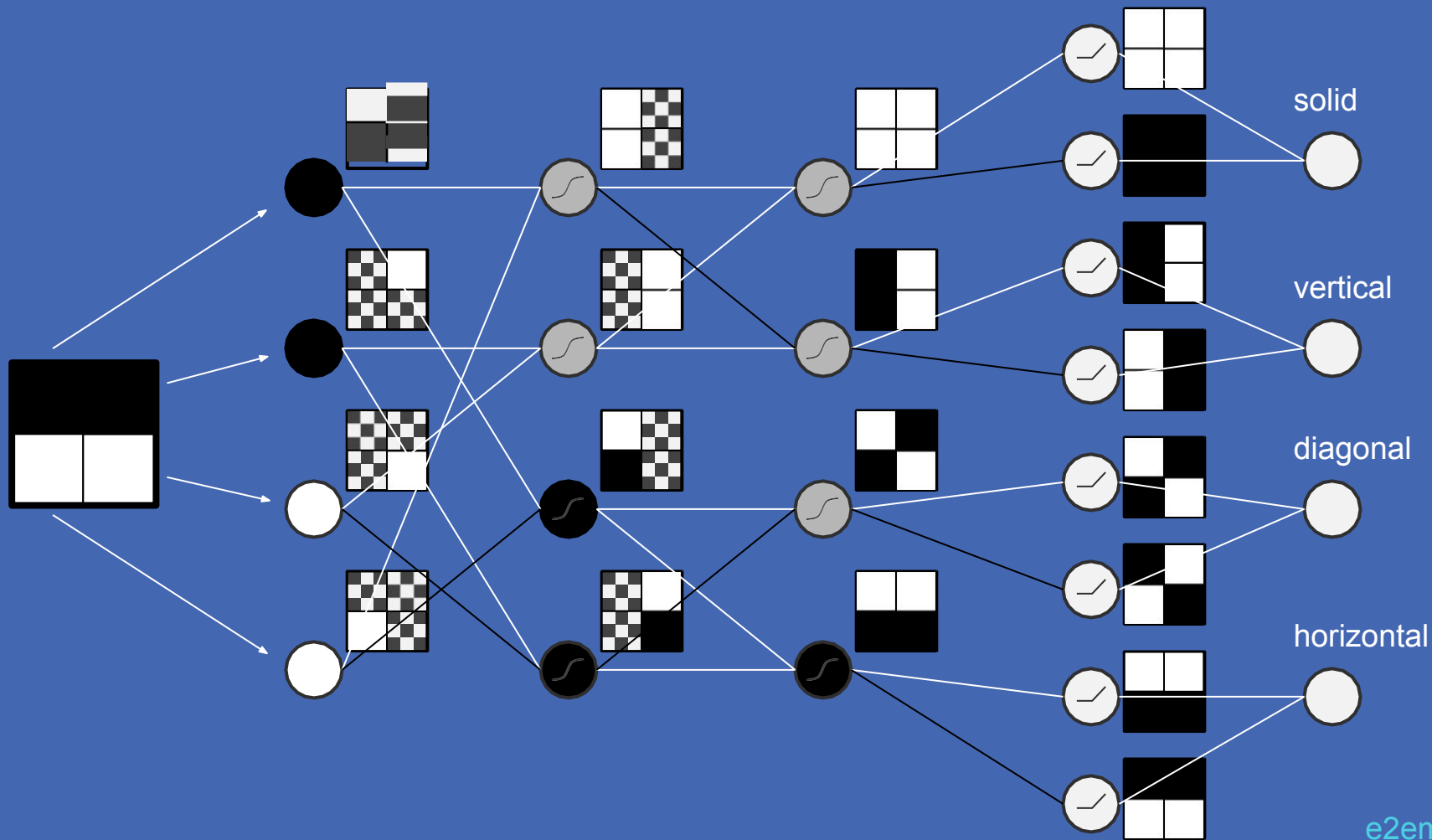
Add an output layer

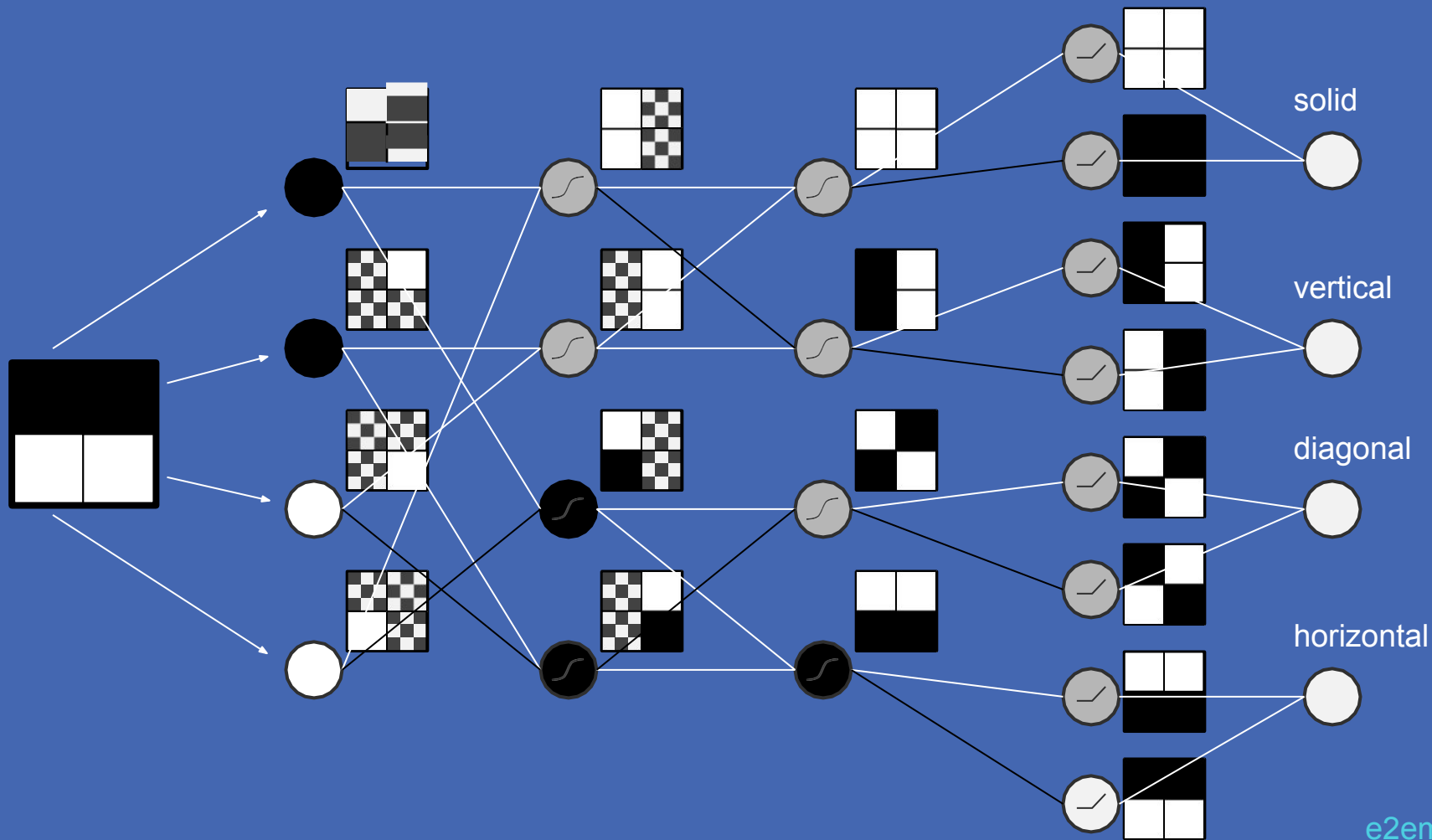


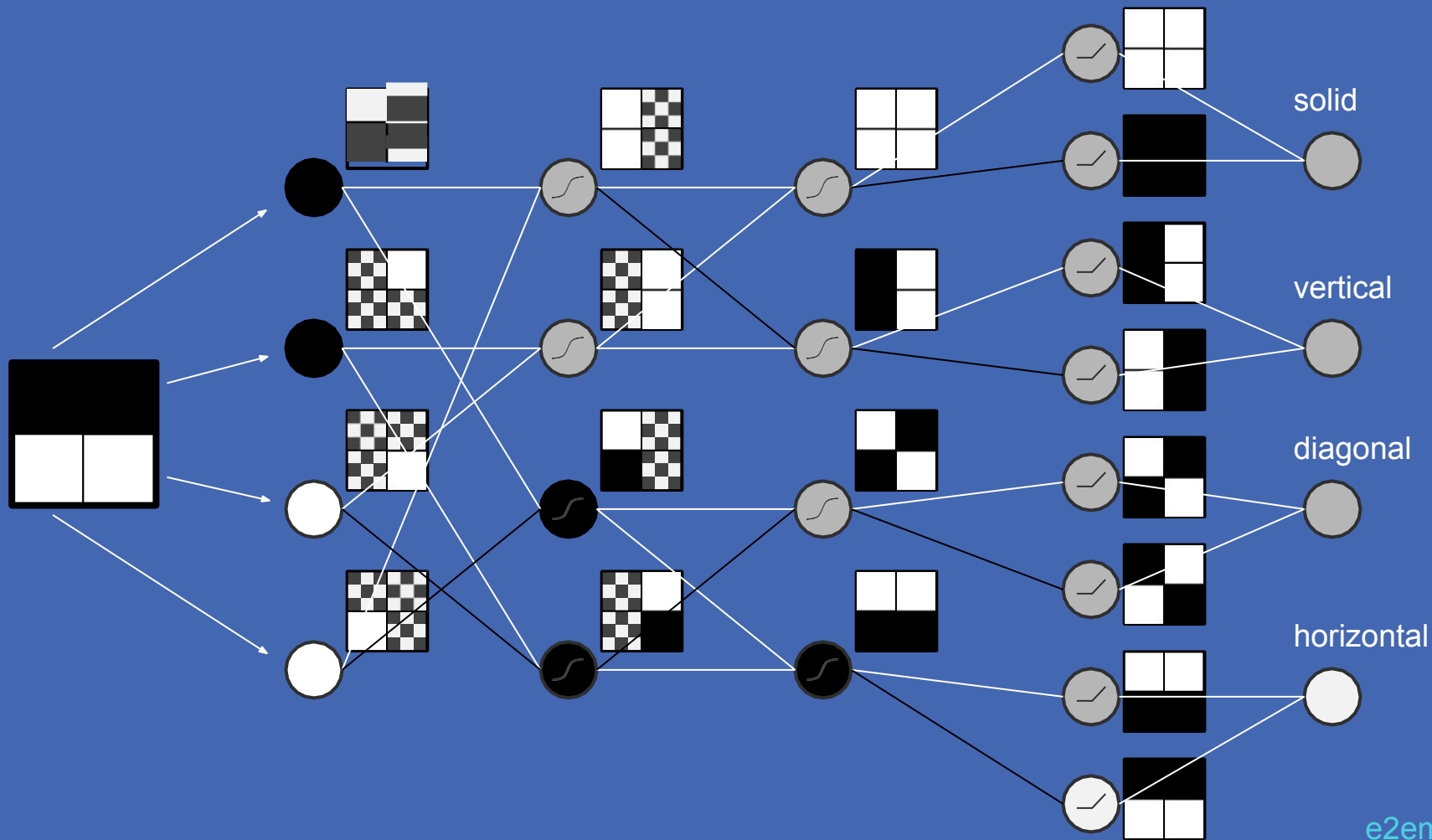


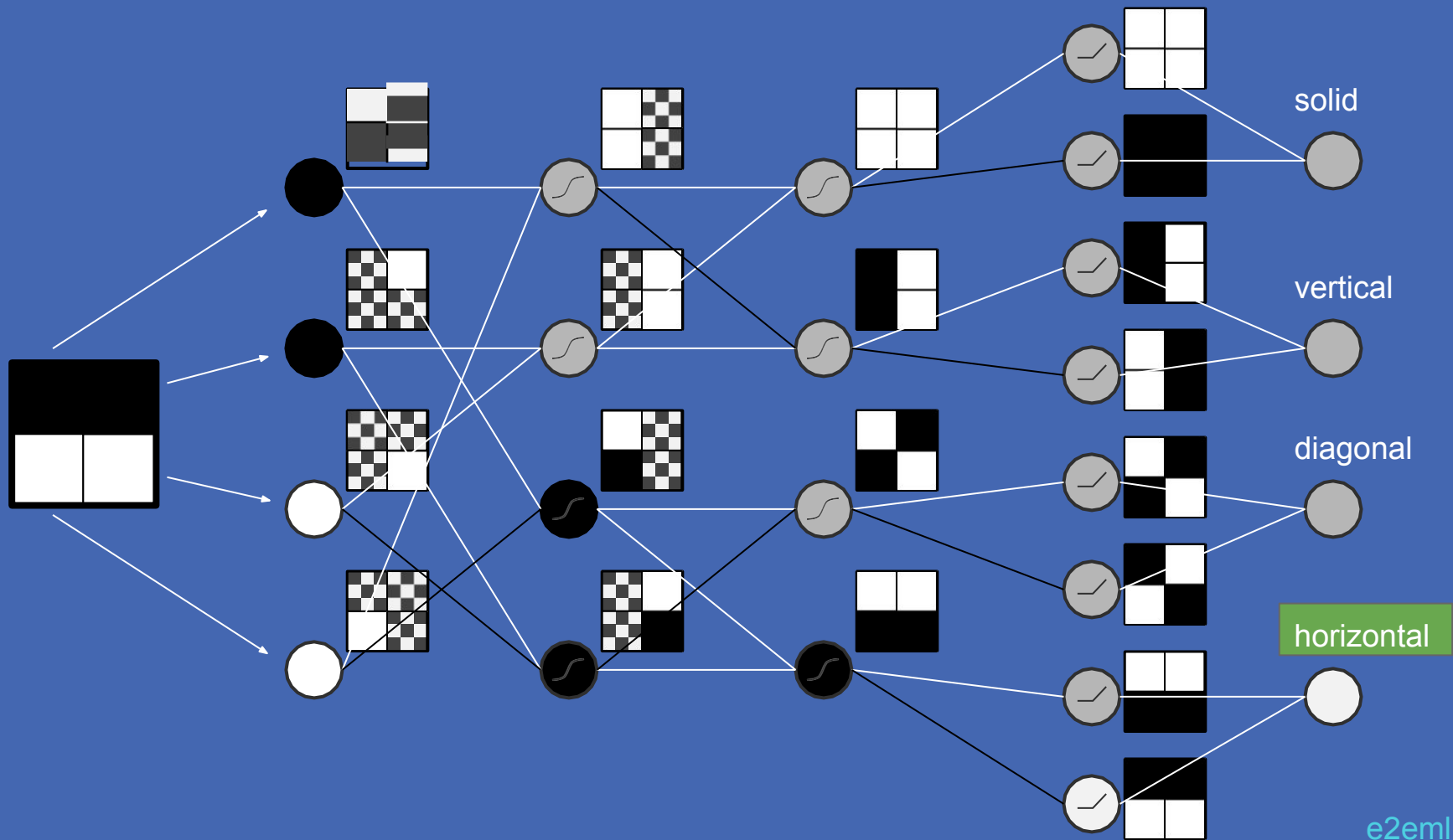




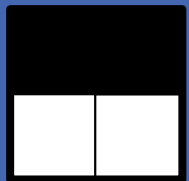








Errors



truth

0.

solid



vertical

0.



diagonal

0.

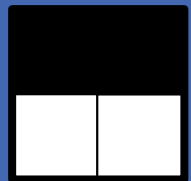


horizontal

1.

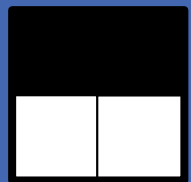


Errors



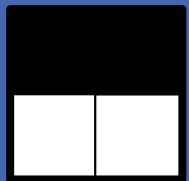
truth	answer	
0.	.5	solid 
0.	.75	vertical 
0.	-.25	diagonal 
1.	-.75	horizontal 

Errors



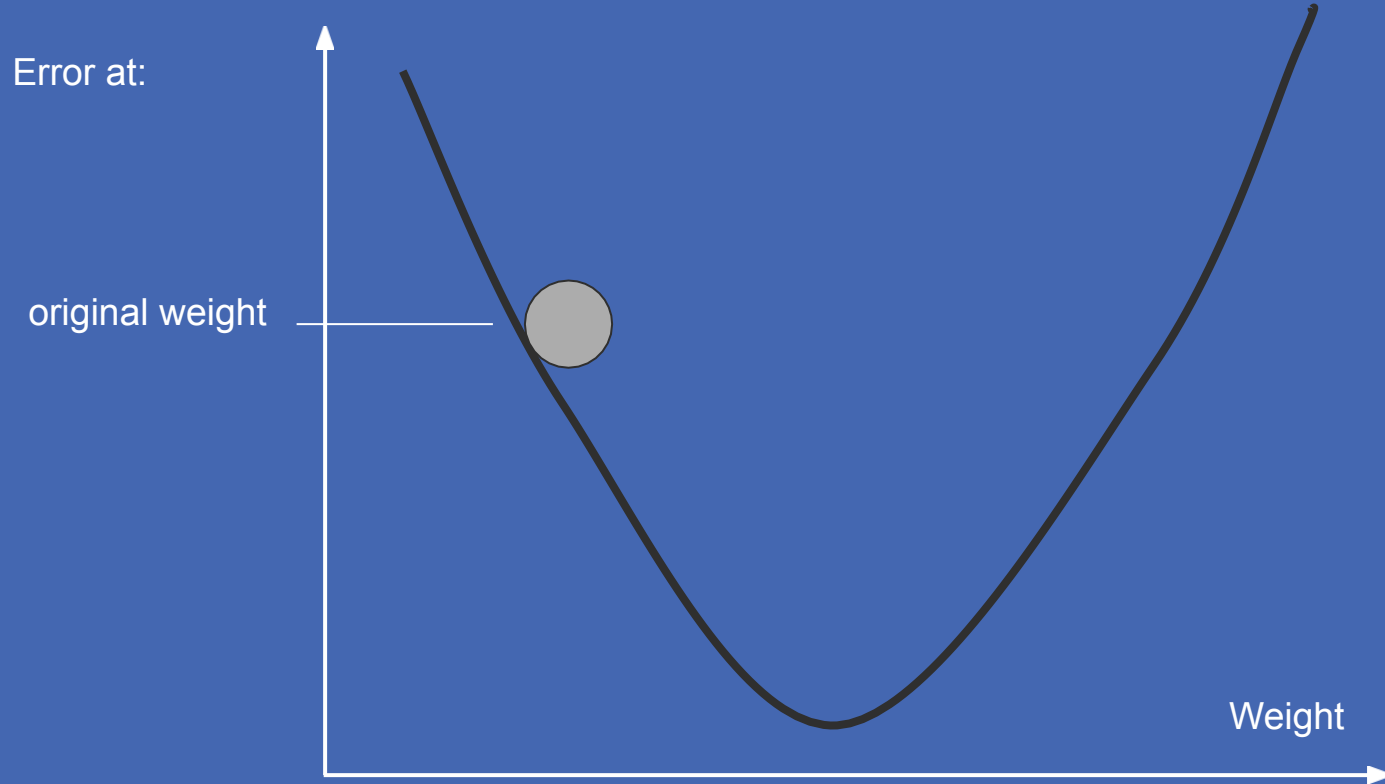
error	truth	answer	
.5	0.	.5	solid 
.75	0.	.75	vertical 
.25	0.	-.25	diagonal 
1.75	1.	-.75	horizontal 

Errors

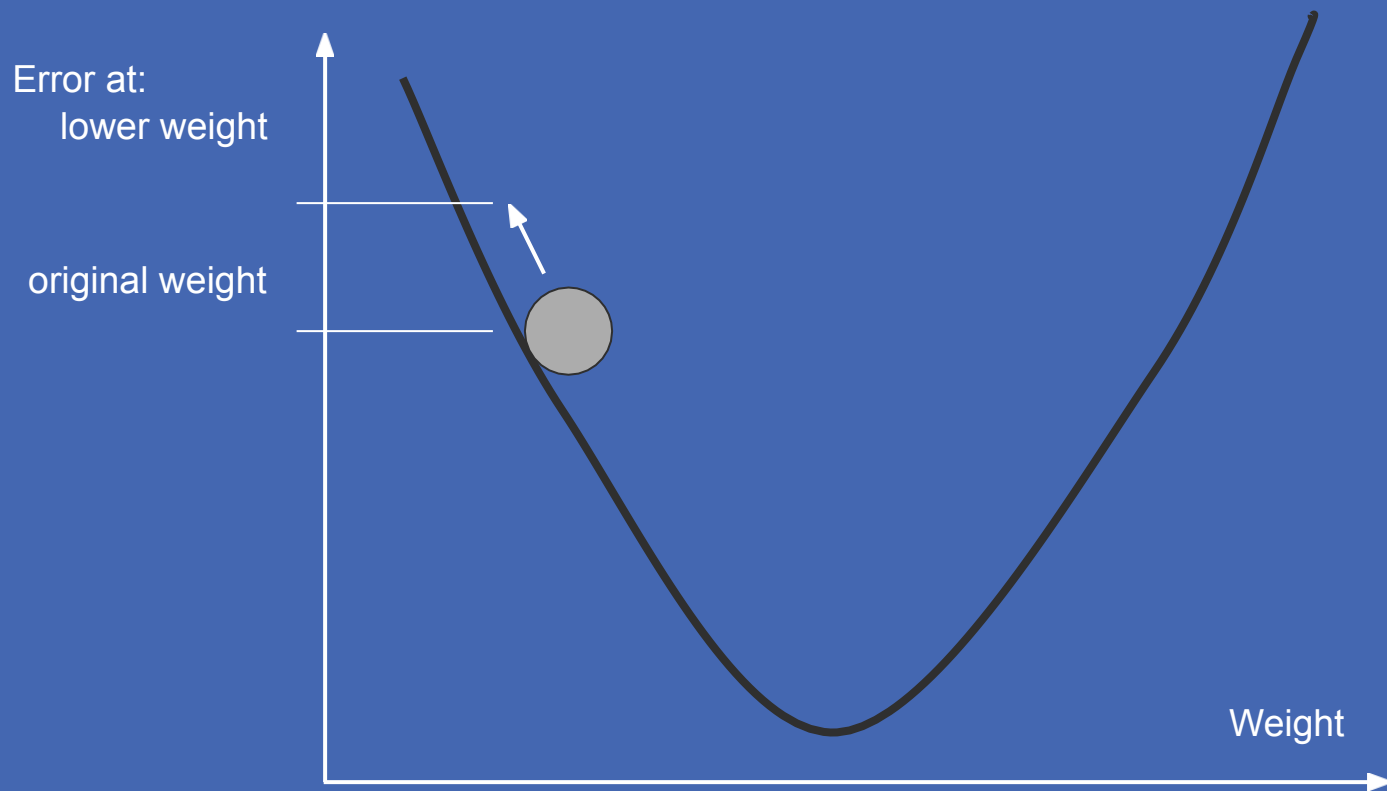


	error	truth	answer	
	.5	0.	.5	solid 
	.75	0.	.75	vertical 
	.25	0.	-.25	diagonal 
	1.75	1.	-.75	horizontal 
total	3.25			

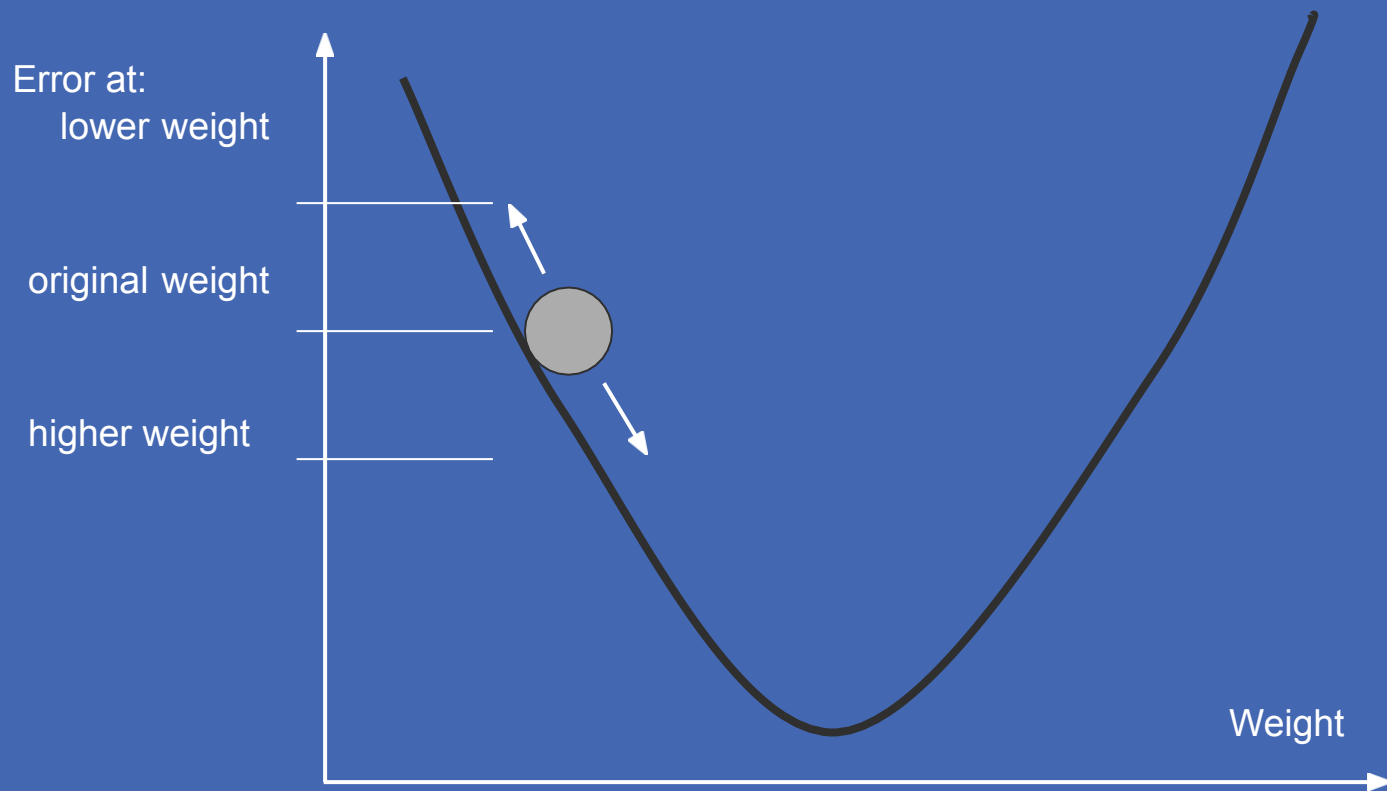
Learn all the weights: Gradient descent



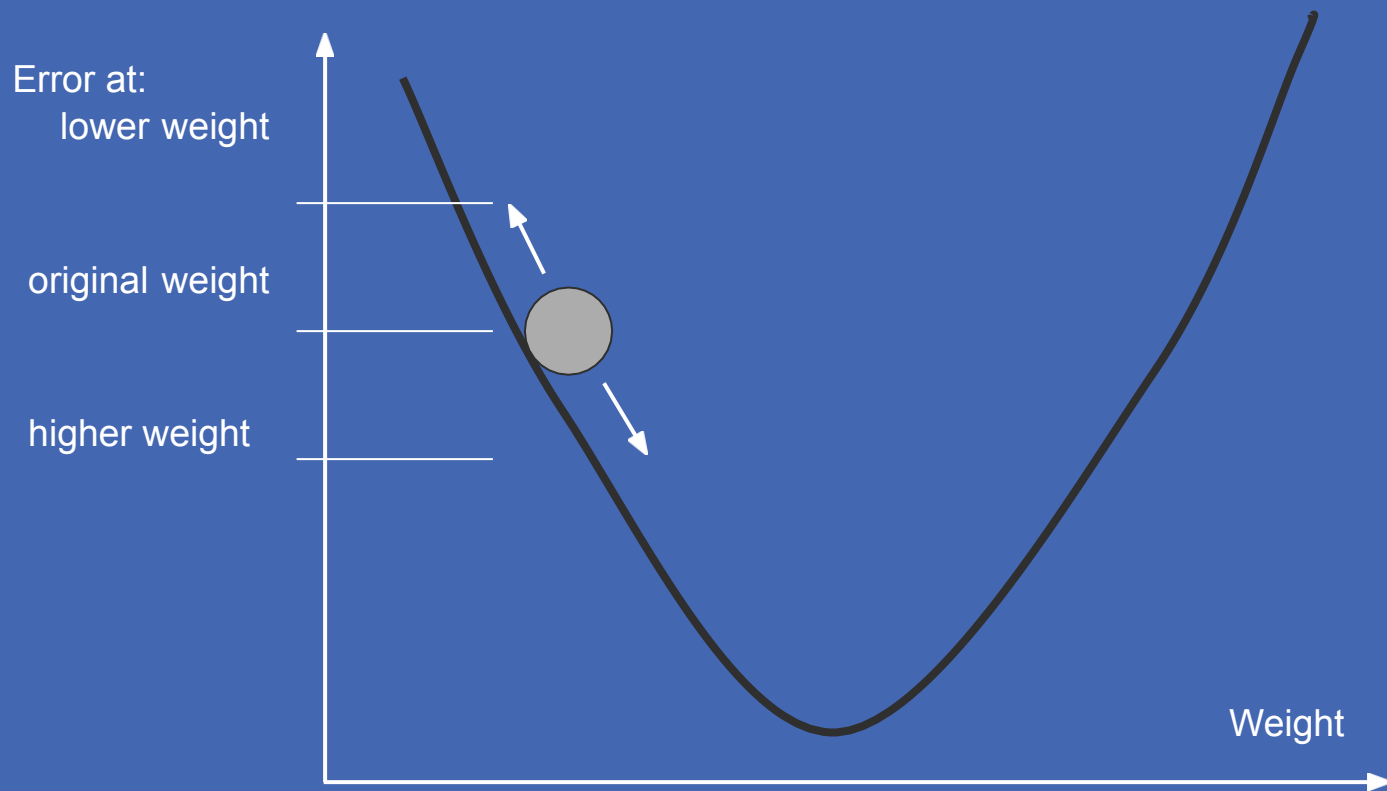
Learn all the weights: Gradient descent



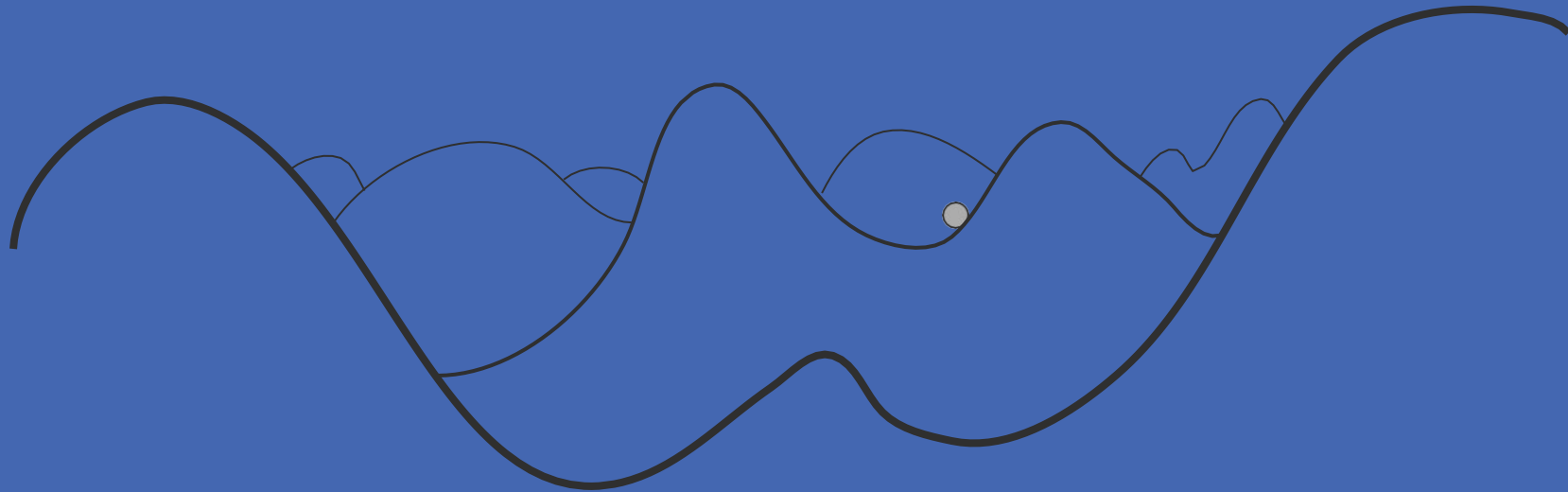
Learn all the weights: Gradient descent



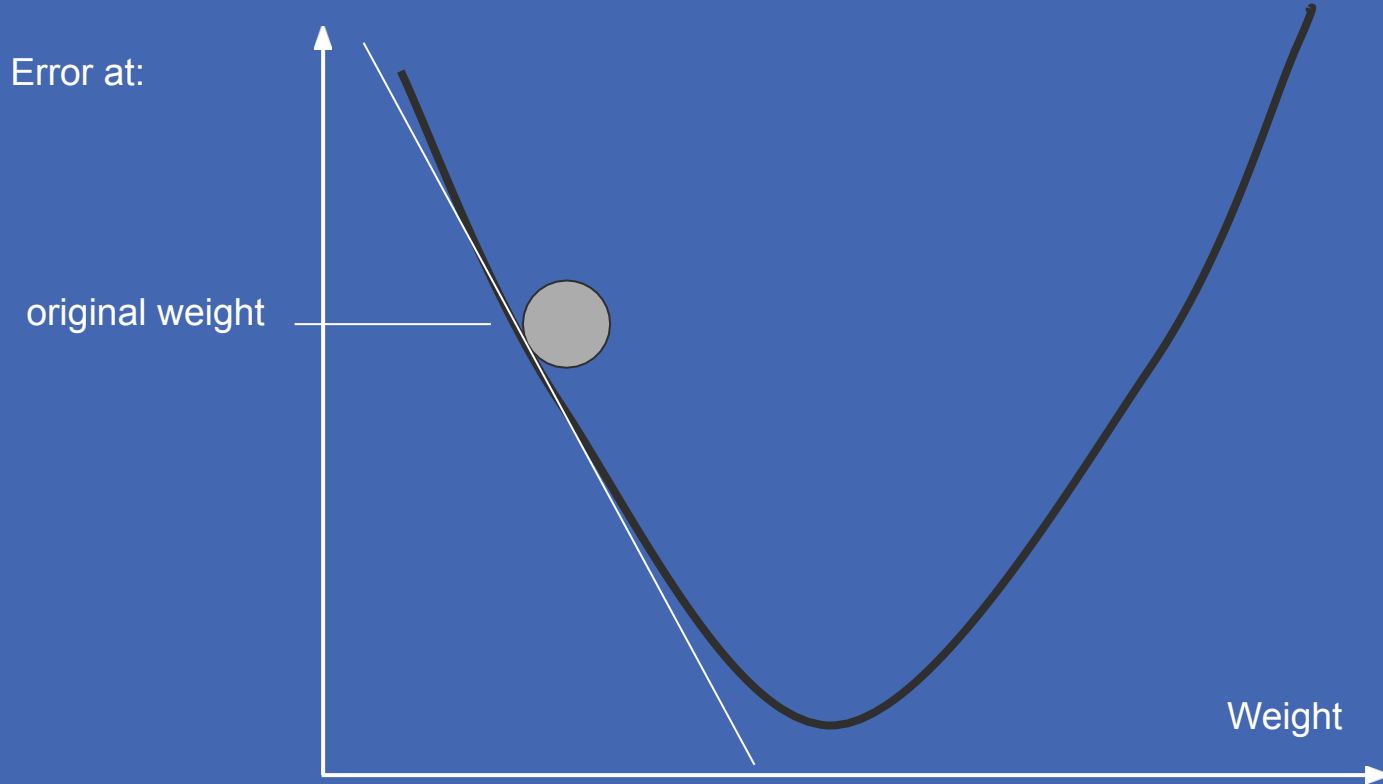
Numerically calculating the gradient is expensive



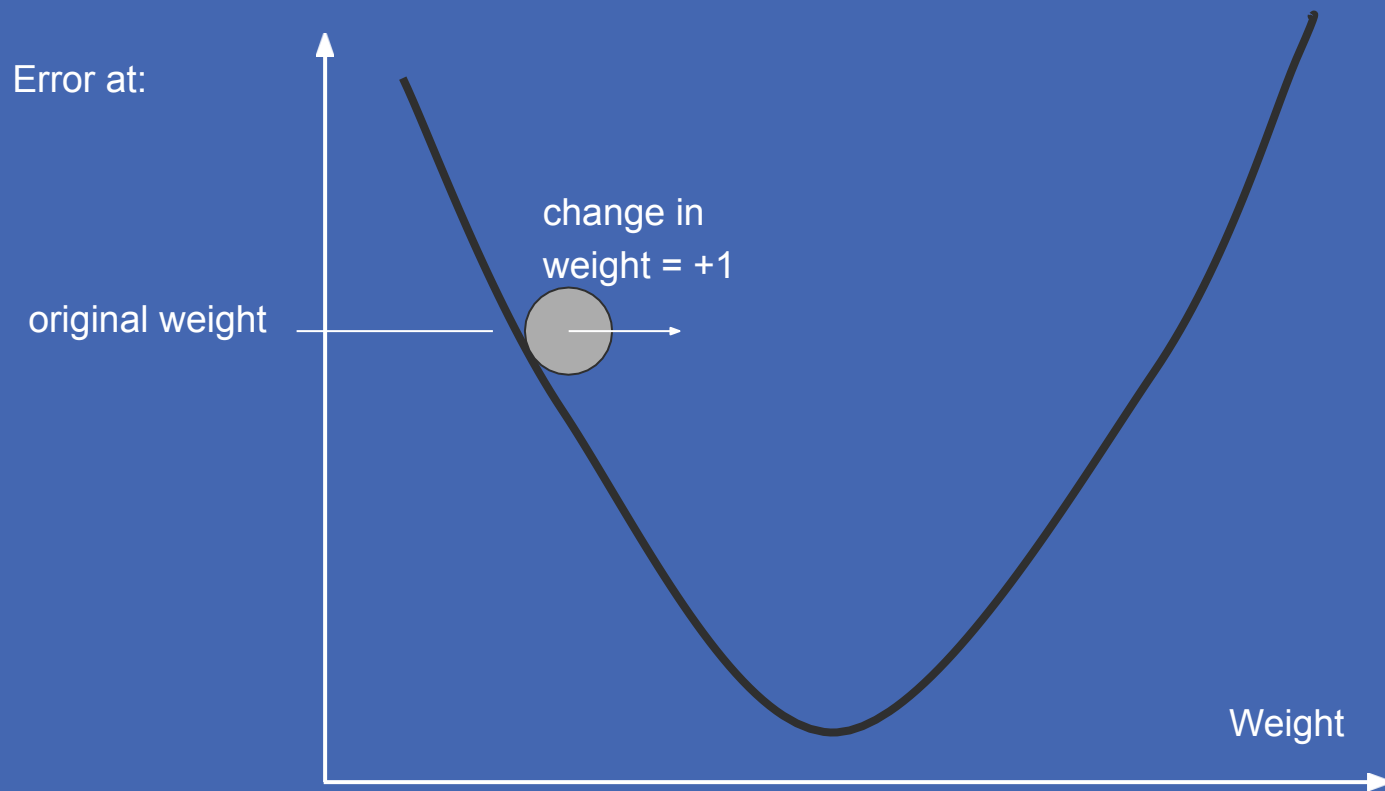
Numerically calculating the gradient is very expensive



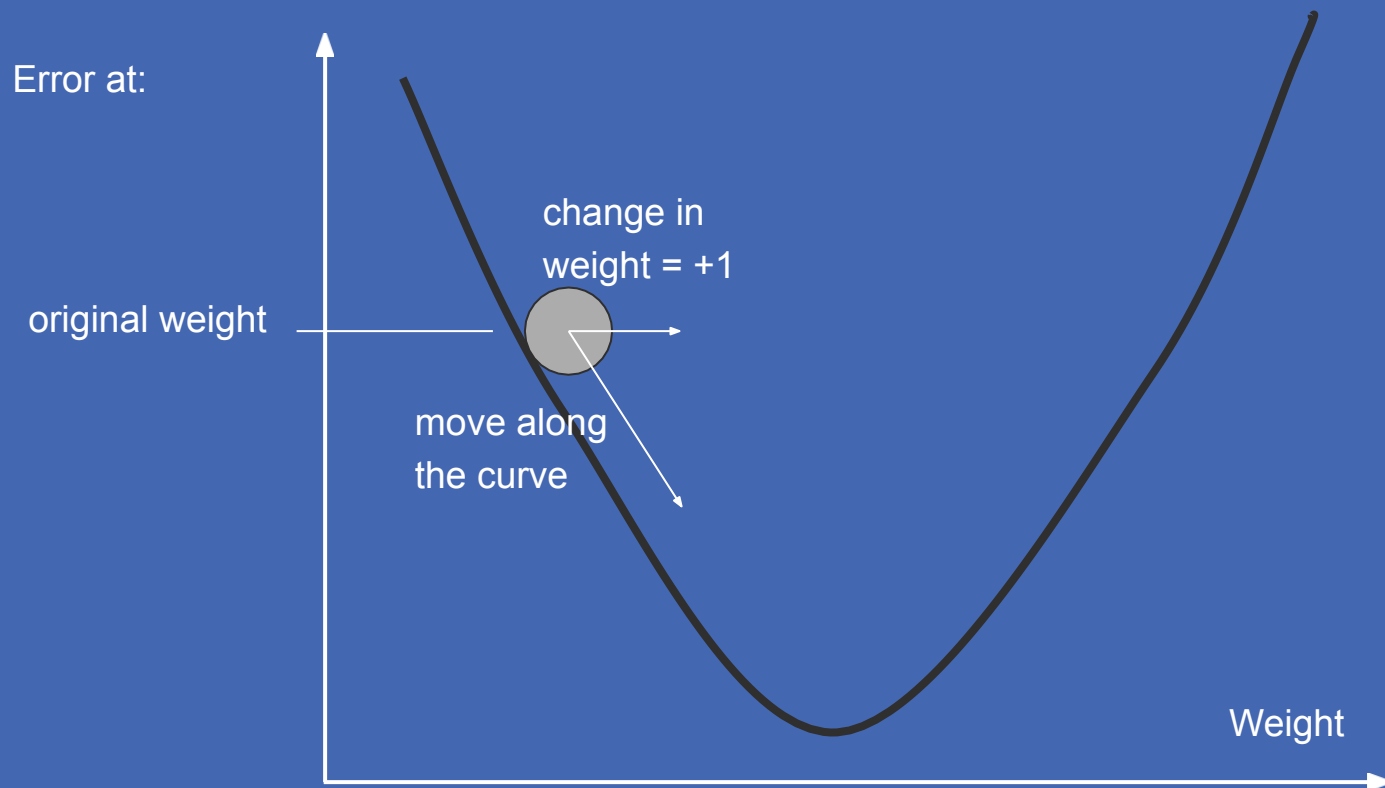
Calculate the gradient (slope) directly



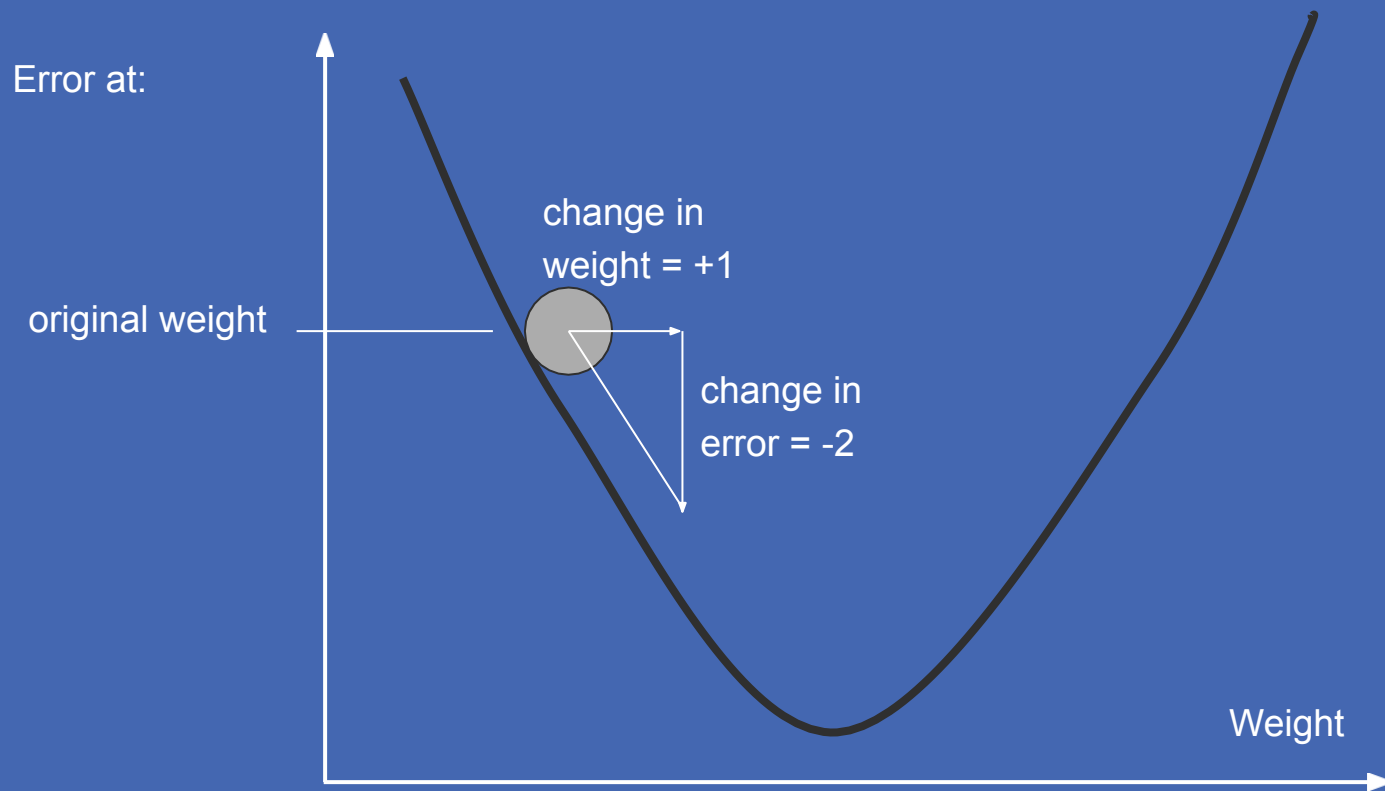
Slope



Slope



Slope



Slope

$$\text{slope} = \frac{\text{change in error}}{\text{change in weight}}$$

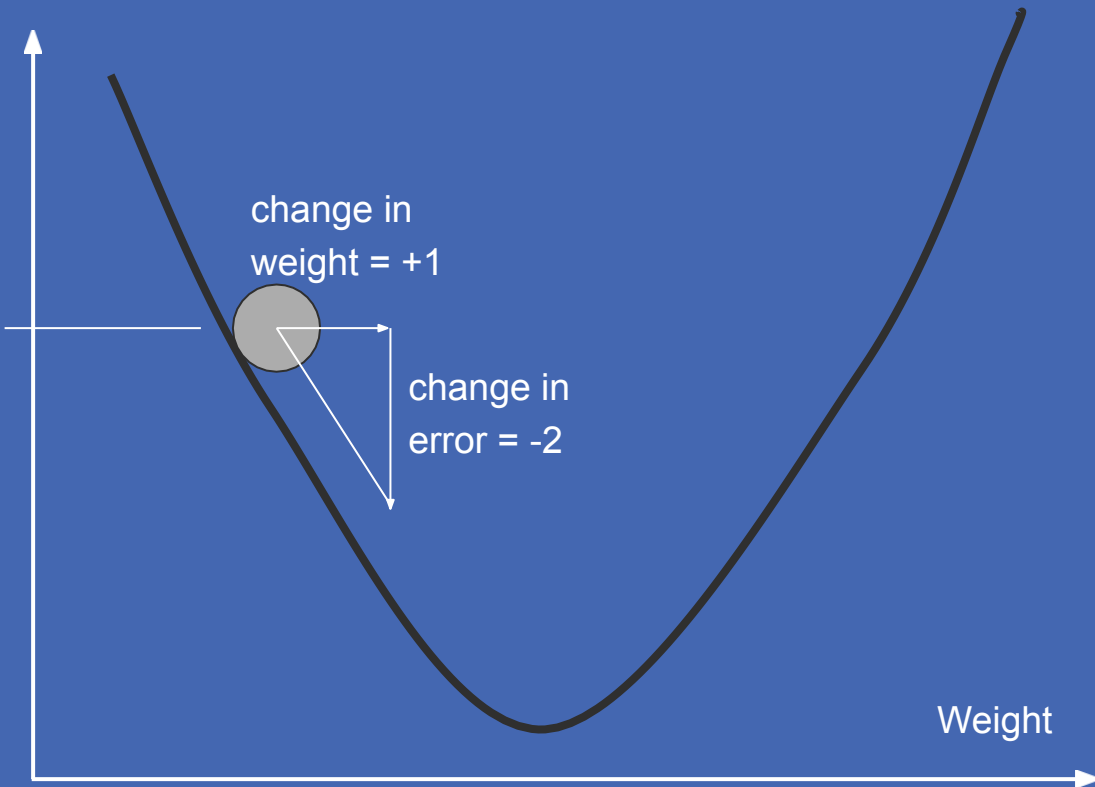
Error at:

original weight

change in
weight = +1

change in
error = -2

Weight



Slope

Error at:

original weight

change in
weight = +1

change in
error = -2

Weight

$$\text{slope} = \frac{\text{change in error}}{\text{change in weight}}$$

$$\begin{aligned} &= \frac{\Delta \text{ error}}{\Delta \text{ weight}} \\ &= \frac{d(\text{error})}{d(\text{weight})} \\ &= \frac{\partial e}{\partial w} \end{aligned}$$

Slope

Error at:

original weight

change in
weight = +1

change in
error = -2

$$\text{slope} = \frac{\text{change in error}}{\text{change in weight}}$$

$$\begin{aligned} &= \frac{\Delta \text{ error}}{\Delta \text{ weight}} \\ &= \frac{d(\text{error})}{d(\text{weight})} \\ &= \frac{\partial e}{\partial w} \end{aligned}$$

$$= \frac{-2}{+1} = -2$$

Weight

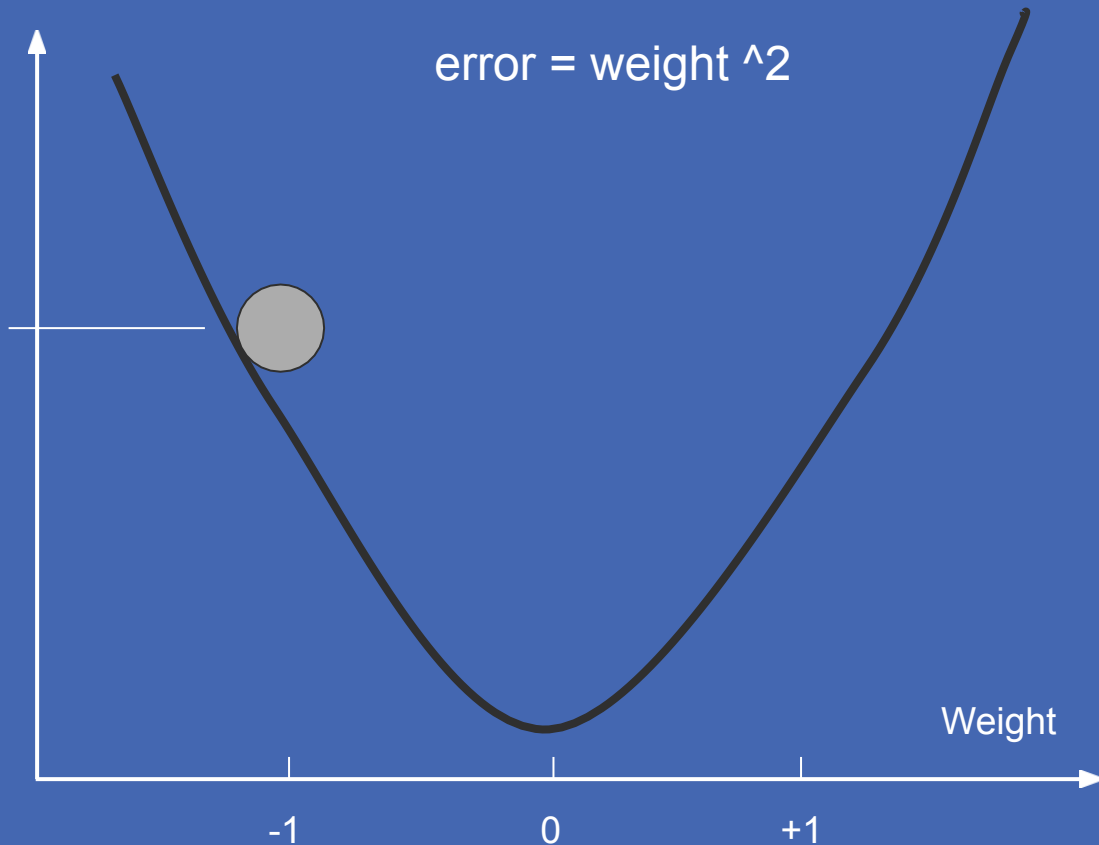
Slope

You have to know your error function.
For example:

$$\text{error} = \text{weight}^2$$

Error at:

original weight



Slope

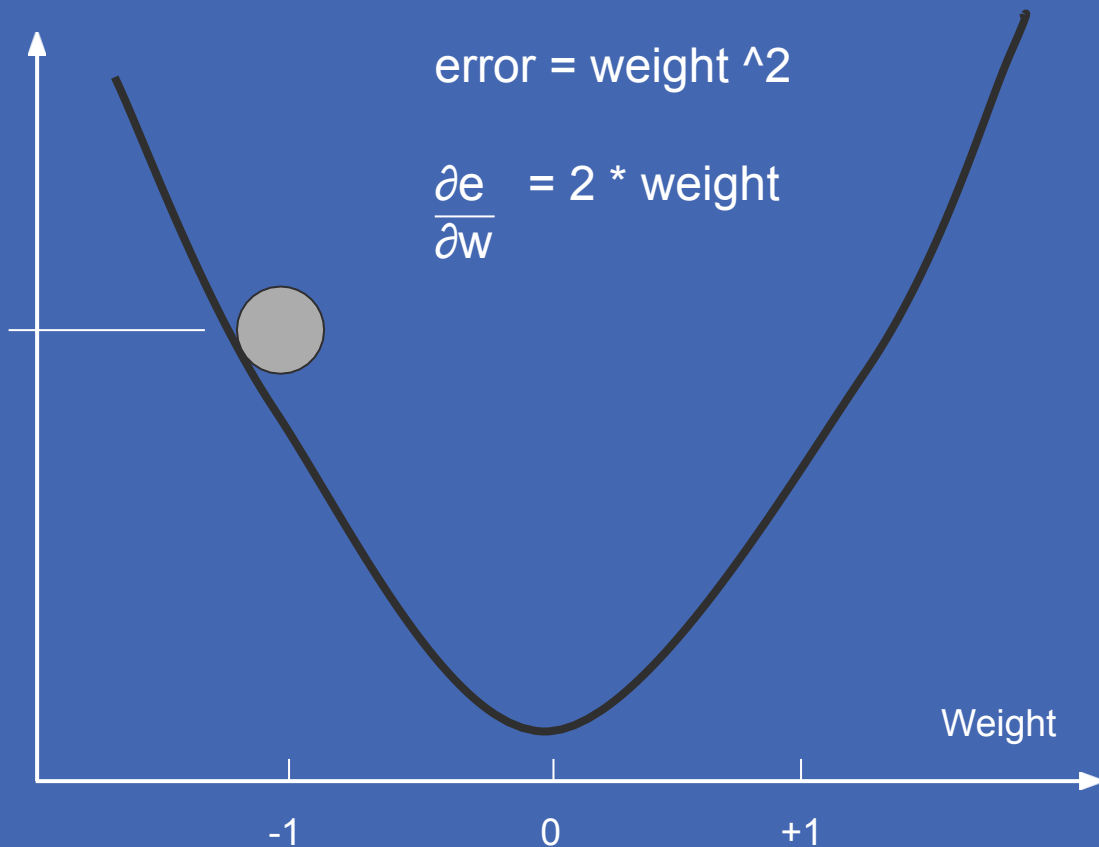
You have to know your error function.
For example:

$$\text{error} = \text{weight}^2$$

$$\frac{\partial e}{\partial w} = 2 * \text{weight}$$

Error at:

original weight



Slope

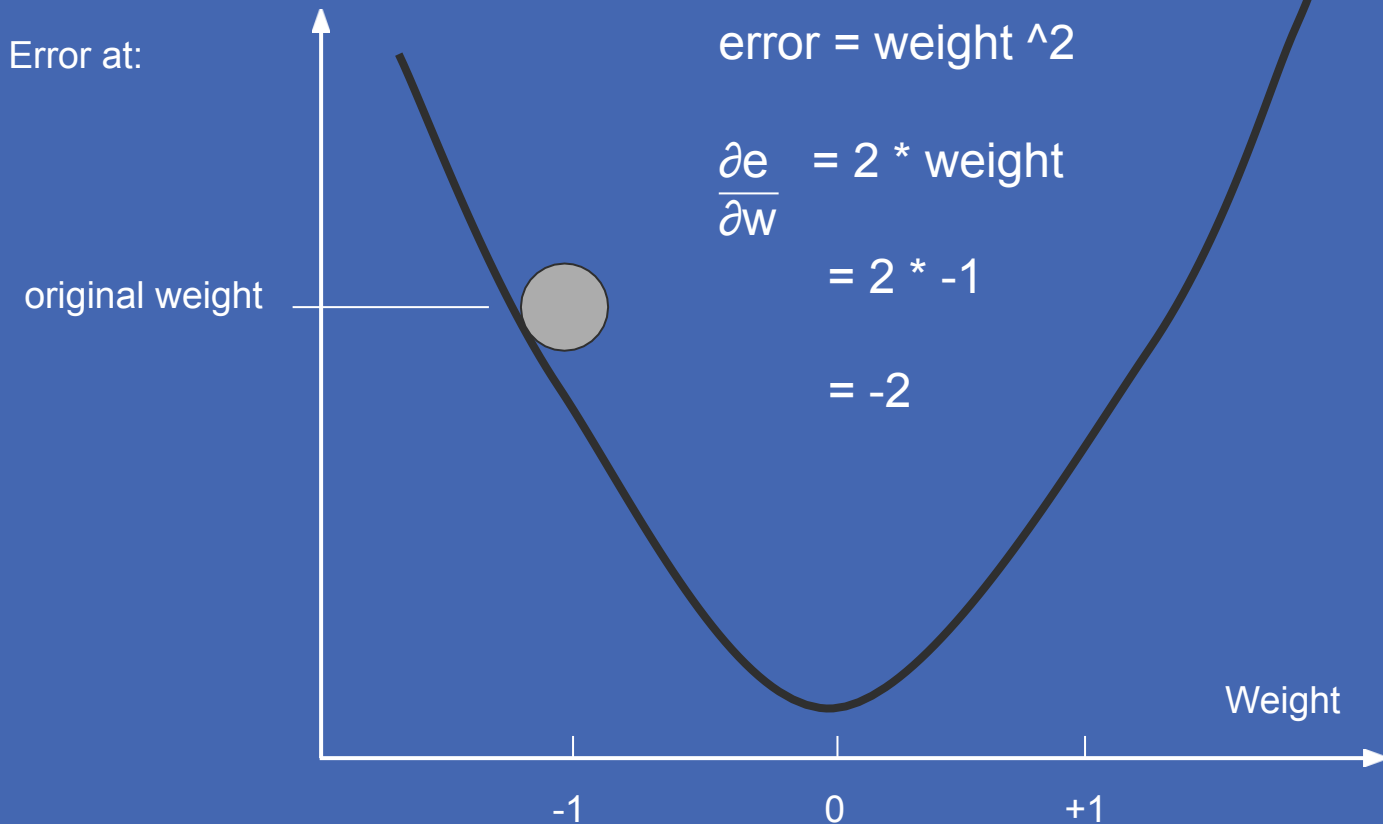
You have to know your error function.
For example:

$$\text{error} = \text{weight}^2$$

$$\frac{\partial e}{\partial w} = 2 * \text{weight}$$

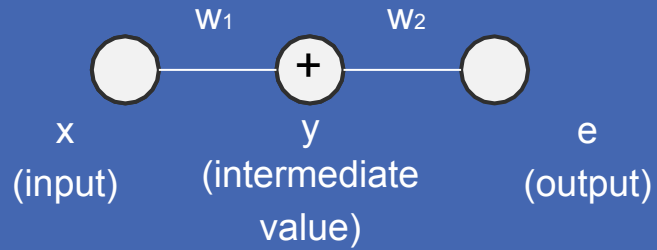
$$= 2 * -1$$

$$= -2$$



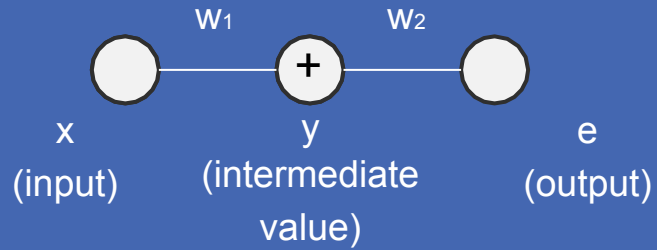
Chaining

$$y = x * w_1$$



Chaining

$$y = x * w_1$$
$$\frac{\partial y}{\partial w_1} = x$$



Chaining

$$y = x * w_1$$

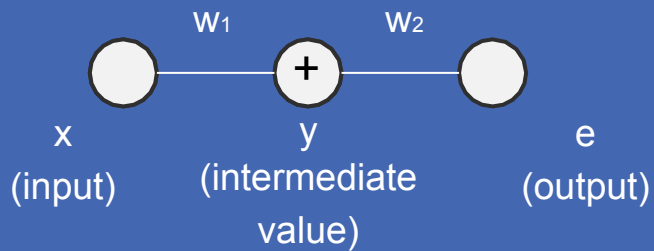
$$\frac{\partial y}{\partial w_1} = x$$

$$\frac{\partial y}{\partial w_1}$$

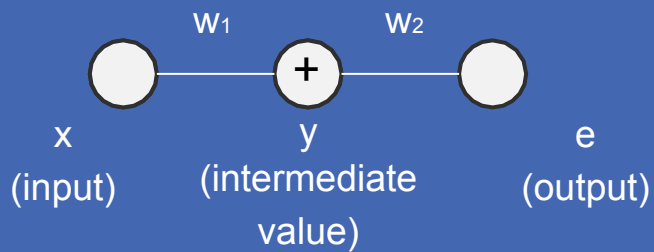
$$e = y * w_2$$

$$\frac{\partial e}{\partial y} = w_2$$

$$\frac{\partial e}{\partial y}$$



Chaining



$$y = x * w_1$$

$$\frac{\partial y}{\partial x} = 1$$

$$\frac{\partial y}{\partial w_1} = x$$

$$e = y * w_2$$

$$\frac{\partial e}{\partial y} = w_2$$

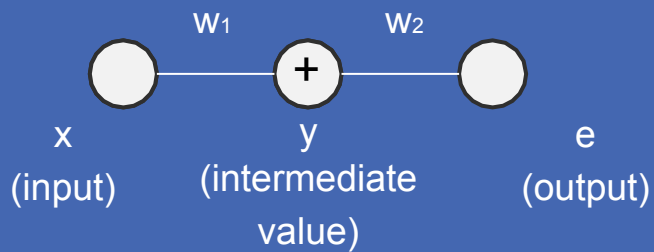
$$\frac{\partial e}{\partial x} = x * w_2$$

$$e = x * w_1 * w_2$$

$$\frac{\partial e}{\partial w_1} = x * w_2$$

$$\frac{\partial e}{\partial w_2} = x * w_1$$

Chaining



$$y = x * w_1$$

$$\frac{\partial y}{\partial w_1} = x$$

$$e = y * w_2$$

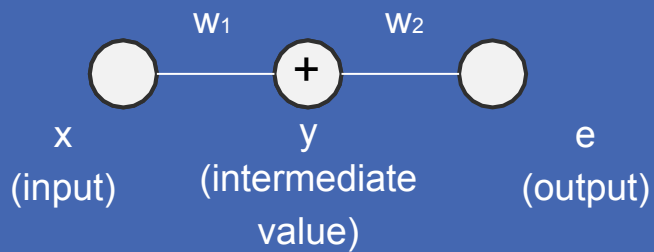
$$\frac{\partial e}{\partial y} = w_2$$

$$e = x * w_1 * w_2$$

$$\frac{\partial e}{\partial w_1} = x * w_2$$

$$\frac{\partial e}{\partial w_1} = \frac{\partial y}{\partial w_1} * \frac{\partial e}{\partial y}$$

Chaining



$$y = x * w_1$$

$$\frac{\partial y}{\partial w_1} = x$$

$$e = y * w_2$$

$$\frac{\partial e}{\partial y} = w_2$$

$$e = x * w_1 * w_2$$

$$\frac{\partial e}{\partial w_1} = x * w_2$$

$$\frac{\partial e}{\partial w_1} = \frac{\partial y}{\partial w_1} * \frac{\partial e}{\partial y}$$

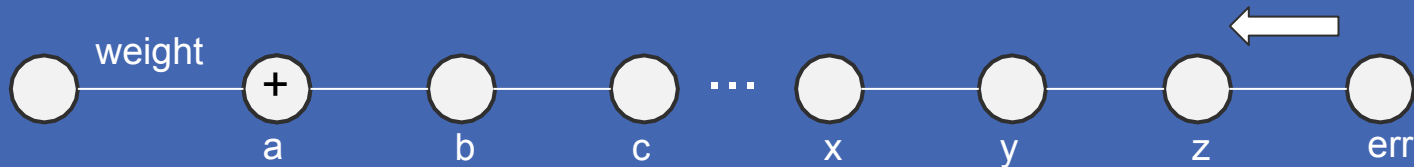
Chaining

$$\frac{\partial \text{err}}{\partial \text{weight}} = \frac{\partial a}{\partial \text{weight}} * \frac{\partial b}{\partial a} * \frac{\partial c}{\partial b} * \frac{\partial d}{\partial c} * \dots * \frac{\partial y}{\partial x} * \frac{\partial z}{\partial y} * \frac{\partial \text{err}}{\partial z}$$



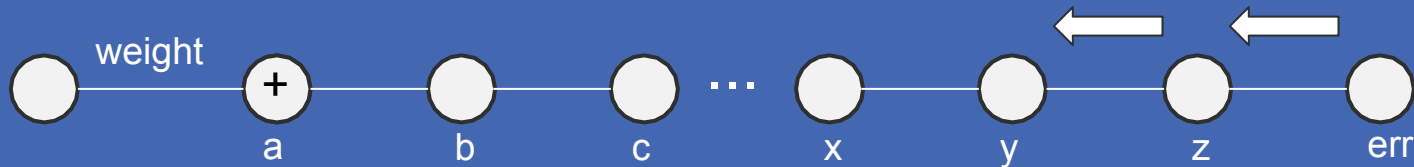
Backpropagation

$$\frac{\partial \text{err}}{\partial \text{weight}} = \frac{\partial a}{\partial \text{weight}} * \frac{\partial b}{\partial a} * \frac{\partial c}{\partial b} * \frac{\partial d}{\partial c} * \dots * \frac{\partial y}{\partial x} * \frac{\partial z}{\partial y} * \frac{\partial \text{err}}{\partial z}$$

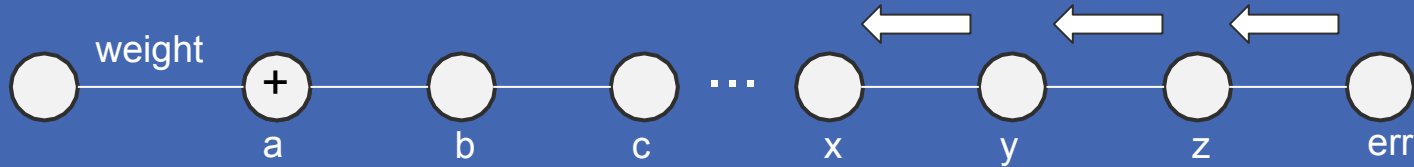
Backpropagation

$$\frac{\partial \text{err}}{\partial \text{weight}} = \frac{\partial a}{\partial \text{weight}} * \frac{\partial b}{\partial a} * \frac{\partial c}{\partial b} * \frac{\partial d}{\partial c} * \dots * \frac{\partial y}{\partial x} * \frac{\partial z}{\partial y} * \frac{\partial \text{err}}{\partial z}$$

Backpropagation

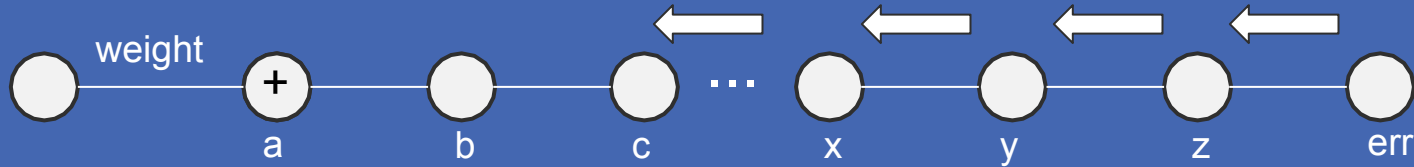
$$\frac{\partial \text{err}}{\partial \text{weight}} = \frac{\partial a}{\partial \text{weight}} * \frac{\partial b}{\partial a} * \frac{\partial c}{\partial b} * \frac{\partial d}{\partial c} * \dots * \frac{\partial y}{\partial x} * \frac{\partial z}{\partial y} * \frac{\partial \text{err}}{\partial z}$$

Backpropagation

$$\frac{\partial \text{err}}{\partial \text{weight}} = \frac{\partial a}{\partial \text{weight}} * \frac{\partial b}{\partial a} * \frac{\partial c}{\partial b} * \frac{\partial d}{\partial c} * \dots * \frac{\partial y}{\partial x} * \frac{\partial z}{\partial y} * \frac{\partial \text{err}}{\partial z}$$

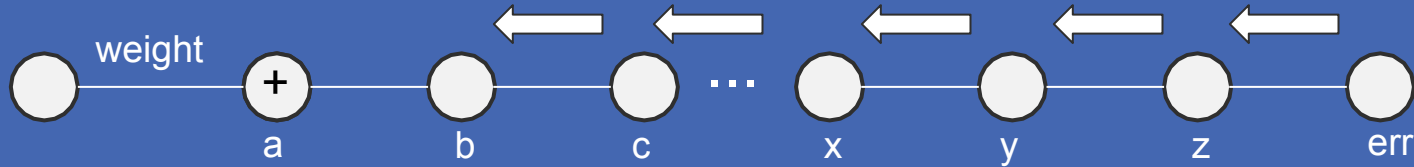
Diagram illustrating the chain rule for backpropagation. The equation shows the derivative of the error with respect to a weight, calculated as the product of derivatives along the path from the error back to the weight. Above the equation, white arrows point left, indicating the direction of backpropagation from the error towards the weight.



Backpropagation

$$\frac{\partial \text{err}}{\partial \text{weight}} = \frac{\partial a}{\partial \text{weight}} * \frac{\partial b}{\partial a} * \frac{\partial c}{\partial b} * \frac{\partial d}{\partial c} * \dots * \frac{\partial y}{\partial x} * \frac{\partial z}{\partial y} * \frac{\partial \text{err}}{\partial z}$$

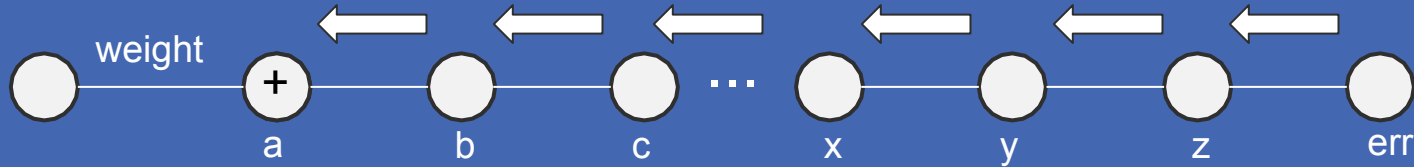
← ← ← ← ←



Backpropagation

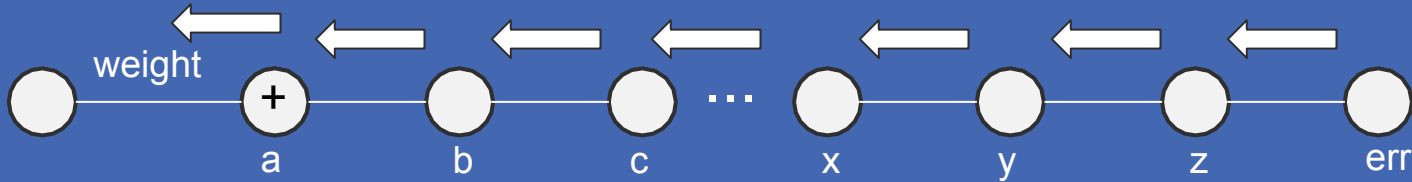
$$\frac{\partial \text{err}}{\partial \text{weight}} = \frac{\partial a}{\partial \text{weight}} * \frac{\partial b}{\partial a} * \frac{\partial c}{\partial b} * \frac{\partial d}{\partial c} * \dots * \frac{\partial y}{\partial x} * \frac{\partial z}{\partial y} * \frac{\partial \text{err}}{\partial z}$$

← ← ← ← ← ←

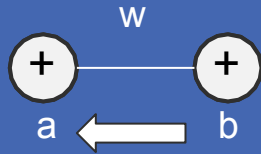


Backpropagation

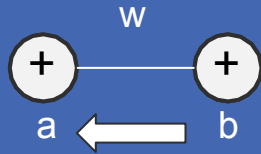
$$\frac{\partial \text{err}}{\partial \text{weight}} = \frac{\partial a}{\partial \text{weight}} * \frac{\partial b}{\partial a} * \frac{\partial c}{\partial b} * \frac{\partial d}{\partial c} * \dots * \frac{\partial y}{\partial x} * \frac{\partial z}{\partial y} * \frac{\partial \text{err}}{\partial z}$$



Backpropagation challenge: weights

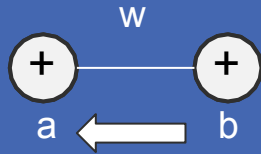


Backpropagation challenge: weights



$$\frac{\partial \text{err}}{\partial a} = \frac{\partial b}{\partial a} * \frac{\partial \text{err}}{\partial b}$$

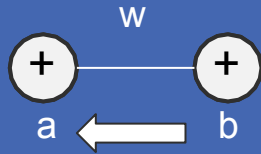
Backpropagation challenge: weights



$$b = wa$$

$$\frac{\partial \text{err}}{\partial a} = \frac{\partial b}{\partial a} * \frac{\partial \text{err}}{\partial b}$$

Backpropagation challenge: weights

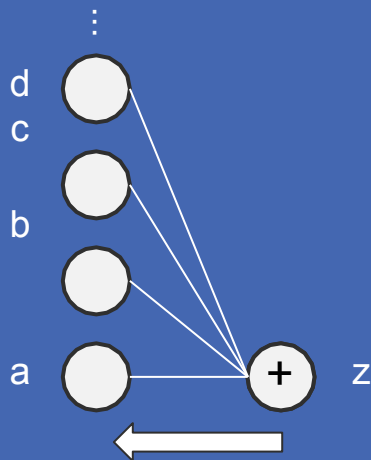


$$\frac{\partial \text{err}}{\partial a} = \frac{\partial b}{\partial a} * \frac{\partial \text{err}}{\partial b}$$

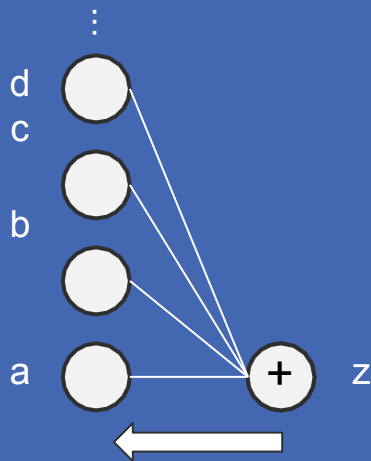
$$b = wa$$


$$\frac{\partial b}{\partial a} = w$$

Backpropagation challenge: sums

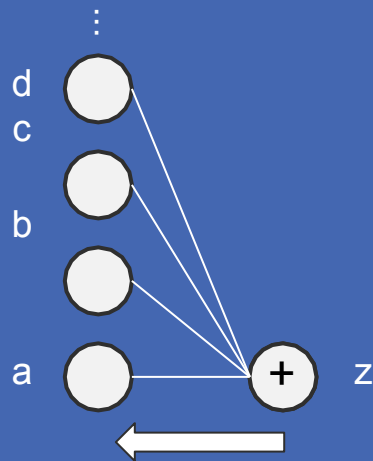


Backpropagation challenge: sums



$$\frac{\partial \text{err}}{\partial a} = \frac{\partial z}{\partial a} * \frac{\partial \text{err}}{\partial z}$$

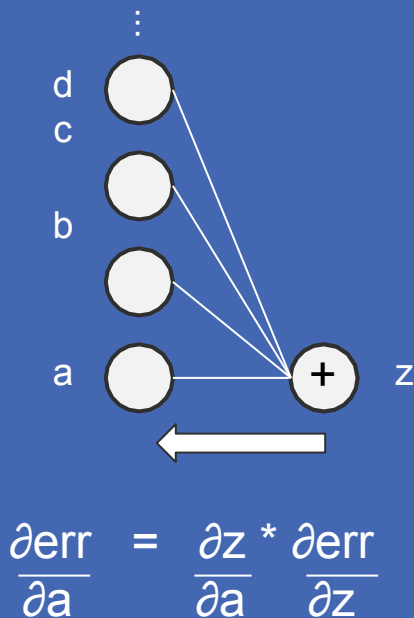
Backpropagation challenge: sums



$$z = a + b + c + d + \dots$$

$$\frac{\partial \text{err}}{\partial a} = \frac{\partial z}{\partial a} * \frac{\partial \text{err}}{\partial z}$$

Backpropagation challenge: sums



$$z = a + b + c + d + \dots$$


$$\frac{\partial z}{\partial a} = 1$$

Backpropagation challenge: sigmoid



$$\frac{\partial \text{err}}{\partial a} = \frac{\partial b}{\partial a} * \frac{\partial \text{err}}{\partial b}$$

Backpropagation challenge: sigmoid

$$b = \frac{1}{1 + e^{-a}}$$



$$\frac{\partial \text{err}}{\partial a} = \frac{\partial b}{\partial a} * \frac{\partial \text{err}}{\partial b}$$

Backpropagation challenge: sigmoid

$$\begin{aligned}b &= \frac{1}{1 + e^{-a}} \\ &= \sigma(a)\end{aligned}$$

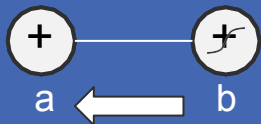


$$\frac{\partial \text{err}}{\partial a} = \frac{\partial b}{\partial a} * \frac{\partial \text{err}}{\partial b}$$

Backpropagation challenge: sigmoid

$$\begin{aligned}b &= \frac{1}{1 + e^{-a}} \\ &= \sigma(a)\end{aligned}$$

Because math is beautiful /
dumb luck:



$$\frac{\partial \text{err}}{\partial a} = \frac{\partial b}{\partial a} * \frac{\partial \text{err}}{\partial b}$$

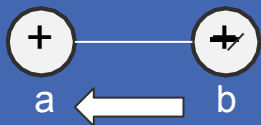

$$\frac{\partial b}{\partial a} = \sigma(a) * (1 - \sigma(a))$$

Backpropagation challenge: ReLU



$$\frac{\partial \text{err}}{\partial a} = \frac{\partial b}{\partial a} * \frac{\partial \text{err}}{\partial b}$$

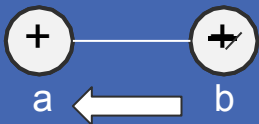
Backpropagation challenge: ReLU



$$\frac{\partial \text{err}}{\partial a} = \frac{\partial b}{\partial a} * \frac{\partial \text{err}}{\partial b}$$

$$\begin{aligned} b &= a, a > 0 \\ &= 0, \text{otherwise} \end{aligned}$$

Backpropagation challenge: ReLU

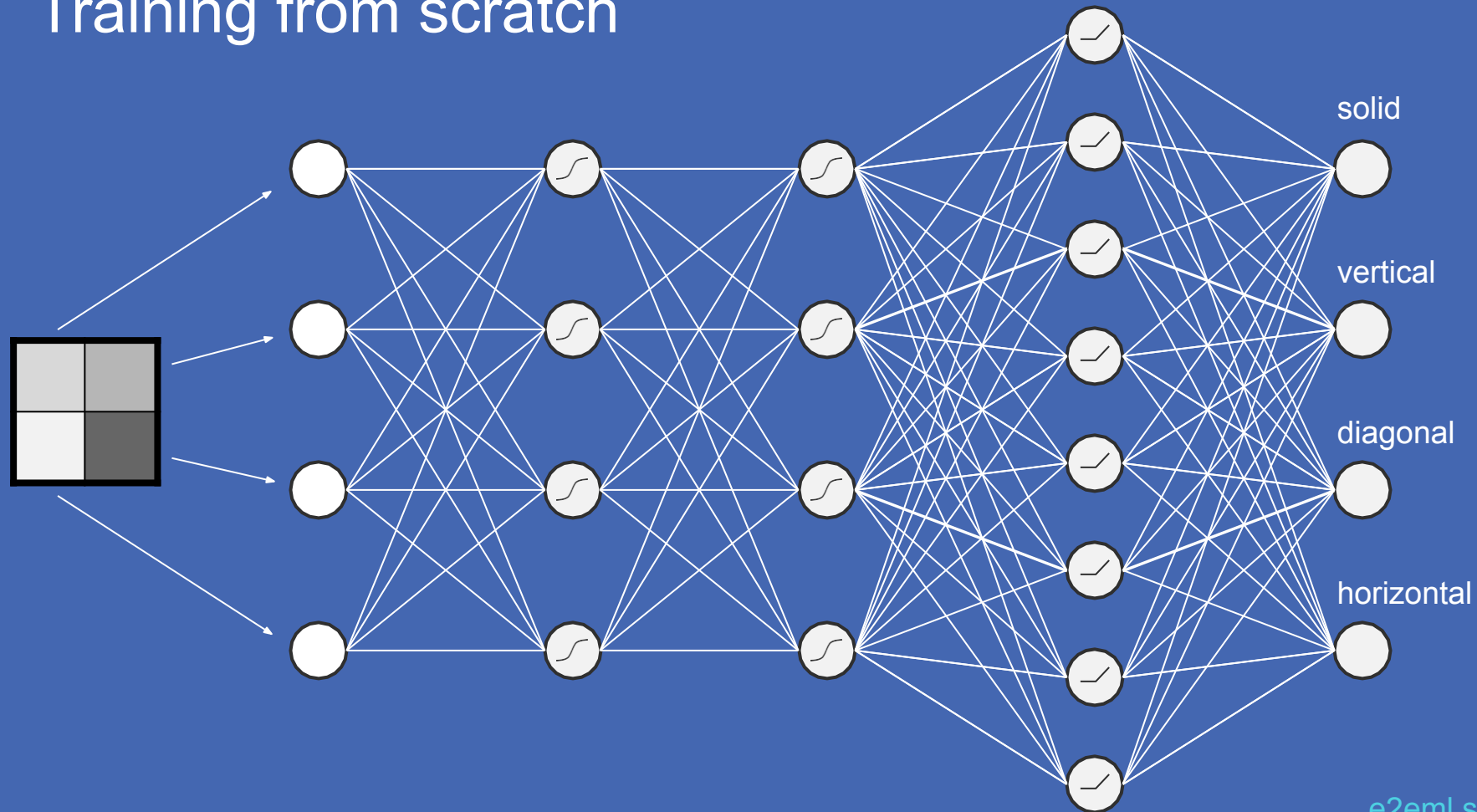


$$\frac{\partial \text{err}}{\partial a} = \frac{\partial b}{\partial a} * \frac{\partial \text{err}}{\partial b}$$

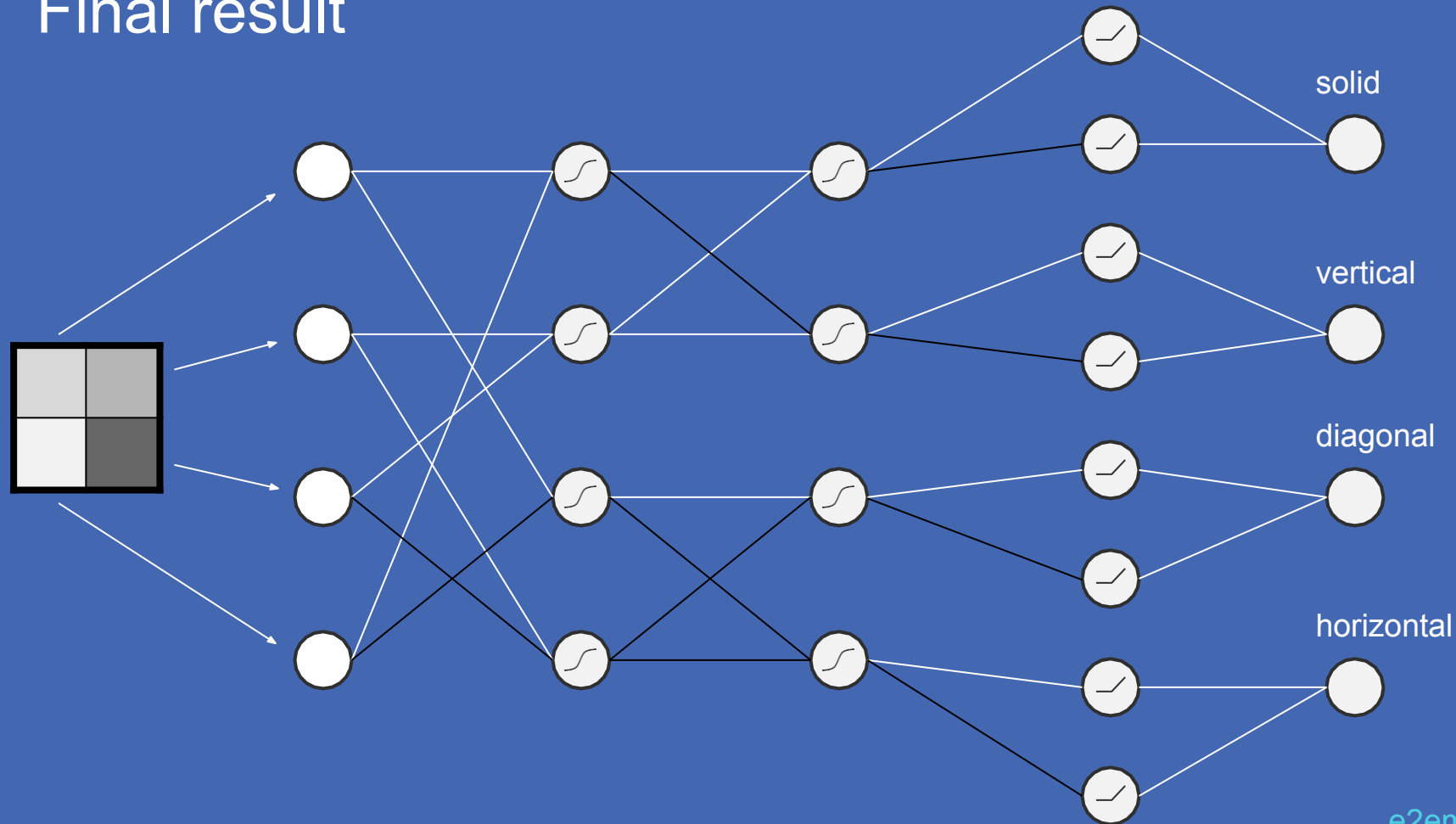
$$\begin{aligned} b &= a, a > 0 \\ &= 0, \text{ otherwise} \end{aligned}$$

$$\frac{\partial b}{\partial a} = \begin{aligned} &1, a > 0 \\ &0, \text{ otherwise} \end{aligned}$$

Training from scratch



Final result



Next Lesson

- ANN in detail : Loss and activation function in depth, B.P., SGD, optimizer
- Intro to Convolutional Neural Network (CNN)

